

ISTRAC-FMS: Chapter 1 — Executive Summary

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 1: Executive Summary	1
Core Problems Solved	1
Guiding Philosophy	1
1.1 Technology Stack	1
1.2 Additions & Gaps Addressed in This Document	2

Chapter 1: Executive Summary

ISTRAC-FMS is a private intranet web application that provides a centralized, permission-controlled interface to manage files and documents stored on network-attached storage. It acts as a UI and governance layer over existing HDD/NAS file trees — the files stay on the servers, the application brings access control, search, audit trails, and approval workflows on top of them.

Core Problems Solved

- Files are currently stored on shared HDD/NAS paths with no controlled access — anyone on the network can read or delete sensitive files.
- There is no audit trail: no record of who accessed what, when, or what changed.
- No version history: overwriting a file loses previous data permanently.
- No workflow: uploads happen ad-hoc with no approval or categorization.
- No search: finding a file requires knowing exactly where it lives on the disk.

Guiding Philosophy

Files are NOT moved out of the HDD. The database stores only metadata and permissions — the actual file bytes remain on existing storage. This preserves existing infrastructure, avoids DB bloat, and keeps the system non-invasive.

1.1 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React + Vite	React 18	Single-page application (SPA)
Styling	Tailwind CSS	v3	Utility-first styling, responsive design

Layer	Technology	Version	Purpose
Backend	Node.js + Express	Node 20 LTS	REST API server + WebSocket
Language	TypeScript	v5 strict	Type safety across full stack
Database	MySQL	8.0+	Metadata, permissions, audit logs
File Storage	HDD / NAS	Existing	Actual file bytes (source of truth)
OS / Host	Red Hat Enterprise Linux	RHEL 8/9	Server OS for hosting
Web Server	Nginx	Latest stable	Reverse proxy + HTTPS termination
Process Mgr	PM2	Latest stable	Node.js lifecycle management
Email (Intra)	Nodemailer + SMTP	Campus SMTP	Approval / notification emails

1.2 Additions & Gaps Addressed in This Document

This v2.1 update adds formal requirements in the following areas identified as missing from v2.0:

- **Security:** CSRF protection, HTTP security headers, file encryption at rest, MFA, CORS policy.
- **Testing:** Unit, integration, and E2E test strategy; CI/CD pipeline.
- **Data Integrity:** Concurrent write handling, file checksums, transactional rollback.
- **Large File Handling:** Chunked / resumable uploads.
- **User Lifecycle:** Password reset flow, password policy enforcement.
- **API Design:** Pagination strategy across all list endpoints.
- **WebSocket Security:** Authentication on WS connections, reconnection strategy.
- **Observability:** Application logging, log rotation, uptime monitoring.
- **Folder Management:** User-level folder creation permissions.
- **Admin Configuration:** Runtime settings panel.
- **Disaster Recovery:** RTO/RPO targets and documented recovery procedure.
- **Environments:** Staging/dev environment strategy.
- **SRS Formality:** Acceptance criteria, traceability, use cases.

Each addition is woven into the relevant chapter and framed as both a gap description and a formal requirement statement (SHALL / SHOULD).

ISTRAC-FMS: Chapter 2 — System Context & Scope

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 2: System Context & Scope	1
2.1 Deployment Environment	1
2.2 What This System Does NOT Do	1
2.3 Scope of This Document	2
2.4 Environments (Gap — New in v2.1)	2
2.5 Stakeholders	2

Chapter 2: System Context & Scope

2.1 Deployment Environment

The system runs entirely on the campus intranet with no external internet exposure. All components — web server, application server, database, and file storage — reside within the private network.

Attribute	Value
Network	Private intranet only — no public internet access
Server OS	Red Hat Enterprise Linux (RHEL 8 or 9)
File Storage	Physical HDD or NAS accessible at mount point <code>/mnt/istrac-data</code>
Database	MySQL 8.0 on same or adjacent intranet server
Email	Campus internal SMTP server (intranet mail only)
Clients	Internal desktops/laptops — Chrome or Firefox browser
Protocol	HTTPS over intranet (self-signed cert acceptable)
Concurrent Users	Up to 200 simultaneous users (intranet scale)

2.2 What This System Does NOT Do

- Does NOT store file bytes in the database — DB holds metadata only.

- Does NOT move or copy files from existing HDD locations.
- Does NOT expose any data to the internet.
- Does NOT replace native OS file access for administrators.
- Does NOT require any cloud infrastructure.

2.3 Scope of This Document

This document covers the complete architecture and requirements from three viewpoints:

- **Frontend:** React component hierarchy, state management, routing, and UI design.
- **Backend:** Express API design, service layer, middleware, and security model.
- **Infrastructure:** Database schema, HDD integration, deployment on RHEL, and scaling path.

2.4 Environments (Gap — New in v2.1)

Gap Description

The v2.0 document describes only a production deployment. There is no mention of development or staging environments, environment parity strategy, or how database schema migrations are validated before reaching production. Without a staging environment, changes are tested in production, increasing the risk of regressions and data corruption.

Formal Requirements

SHALL: The project SHALL maintain three named environments: **development** (local developer machines), **staging** (a server-side environment mirroring production configuration), and **production**.

SHALL: The staging environment SHALL be configured identically to production (same OS, same Node.js version, same Nginx config, same MySQL version) with a separate database and a copy of anonymized/sanitized file metadata.

SHALL: All database schema migrations SHALL be applied and validated in staging before being applied to production.

SHALL: Environment-specific configuration SHALL be managed via separate `.env.development`, `.env.staging`, and `.env.production` files. No environment-specific values SHALL be hardcoded.

SHOULD: A CI/CD pipeline SHOULD automatically deploy passing builds to staging. Promotion to production SHALL require a manual approval step.

2.5 Stakeholders

Role	Responsibility
Super Admin	System owner — full control, onboards Dept Admins
Department Admin	Manages members and files within their department
Department Member	Day-to-day file access and uploads
Read-Only Guest	External or limited-access viewer
ISTRAC IT Team	Server maintenance, OS patching, HDD management
Campus IT	Provides SMTP relay, AD/LDAP directory, network

ISTRAC-FMS: Chapter 3 — System Architecture

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 3: System Architecture	1
3.1 High-Level Architecture	1
3.2 Backend: Clean Architecture Layers	2
3.3 Frontend: Component Architecture	2
3.4 Data Flow: File Upload	3
3.5 Transactional Integrity (Gap — New in v2.1)	4
3.6 Concurrent Write Handling (Gap — New in v2.1)	4

Chapter 3: System Architecture

3.1 High-Level Architecture

The system follows a three-tier architecture: a React SPA client tier, a Node.js/Express application tier, and a MySQL + HDD data tier. All tiers communicate within the campus intranet.

INTRANET NETWORK

[BROWSER CLIENT]	HTTPS(443)	[NGINX Reverse Proxy]
React SPA		- HTTPS termination
TailwindCSS		- Static file serving
React Query		- Proxy /api/* → :3000
Zustand		

[NODE.JS / EXPRESS (:3000)]
REST API | WebSocket | HDD Sync

[Service Layer]
AuthSvc | FileSvc | UserSvc
AuditSvc | EmailSvc | SearchSvc

[MySQL DB]	[HDD/NAS]
Users, Permissions	/mnt/istrac-data/
Metadata, Audit Logs	(actual bytes)

3.2 Backend: Clean Architecture Layers

The Node.js backend is organized into four distinct layers, following Clean Architecture principles. Business logic is not tangled with HTTP handling or database queries.

Layer	Folder	Responsibility
Controller (API)	<code>src/api/</code>	Thin HTTP handlers — parse request, call service, return response. No business logic.
Service (Business)	<code>src/services/</code>	All business rules, orchestration between DB and HDD. Core logic lives here.
Repository (Data)	<code>src/repositories/</code>	SQL queries only. No logic. Returns typed results to services.
Middleware	<code>src/middleware/</code>	Auth (JWT), RBAC (role check), Audit (auto-log), HDD availability check.

Key Backend Services

- **AuthService** — JWT generation, refresh token rotation, password hashing (bcrypt cost 12), forced logout.
- **FileService** — orchestrates HDD reads/writes and DB metadata updates for every file operation.
- **HddService** — low-level file system I/O abstraction (read, write, stream, move to trash, version archival).
- **HddSyncService** — daemon that walks the HDD tree every 15 minutes and reconciles it with the DB.
- **AuditService** — appends immutable audit records for every state-changing operation.
- **EmailService** — sends approval/rejection/notification emails via campus SMTP using Node-mailer.
- **SearchService** — executes MySQL FULLTEXT queries scoped to the user's accessible departments.

3.3 Frontend: Component Architecture

The React frontend follows Atomic Design — components are built from atoms (Button, Badge) to molecules (FileCard) to organisms (FileGrid, Sidebar) to pages. Code-splitting via `React.lazy()` keeps the admin bundle separate from the user bundle.

Module	Key Components	Notes
Auth / Public	Landing.tsx, Login.tsx, Register.tsx	No auth required — CMS-driven content
Layout	AppShell, Sidebar, Topbar, PublicLayout	Wraps all protected pages
File Browser	FileGrid, FileList, FileCard, FileUploadModal, FilePreviewModal	Core file interaction UI
Department	DeptCard, CreateDeptModal, DeptBreadcrumb	Department navigation
Admin	UserManagement, ApprovalQueue, DeptManager, AuditLog, CmsEditor	Lazy-loaded, RBAC-gated
Hooks	useFileSystem, usePermissions, useSearch, useAuditLog	Encapsulate API calls
State	React Query (server state), Zustand (UI state), React Hook Form (forms)	No Redux

3.4 Data Flow: File Upload

The following shows the complete data flow for a file upload operation:

User selects file in browser

React validates: file size < configured limit, extension in allowlist

POST /api/v1/files/upload (multipart/form-data)

```
auth.middleware      → validates JWT access token
rbac.middleware       → checks user has WRITE access to target department
hdd.middleware        → verifies HDD mount is available
```

FileController → calls FileService.upload()

FileService:

1. HddService.validateMagicBytes(buffer) → check real file type
2. HddService.generateChecksum(buffer) → SHA-256 hash stored in DB ← NEW v2.1
3. HddService.writeFile(deptPath, buffer) → write bytes to HDD
4. FileRepo.insertNode(metadata) → DB record created (in transaction) ← NEW v2.1

- 5. `AuditService.log('FILE_UPLOAD', ...)` → immutable audit entry
- 6. `WebSocket.broadcast(deptId, 'new_file')` → real-time notification to dept members

201 Created response → React Query cache invalidated → UI updates

3.5 Transactional Integrity (Gap — New in v2.1)

Gap Description

The v2.0 upload flow writes to HDD first, then inserts a DB record. If the DB insert fails after the HDD write succeeds, the file exists on disk with no corresponding metadata record — it will be flagged as UNREGISTERED by the next sync run but will not be accessible to users. Conversely, if HDD write fails after DB insert, users see a ghost record. There is no rollback or compensation logic defined.

Formal Requirements

SHALL: The FileService SHALL implement a compensation pattern: if the DB insert fails after an HDD write, the HDD file SHALL be immediately deleted and the error returned to the client.

SHALL: If the HDD write fails, no DB record SHALL be created. The error SHALL be returned to the client with an appropriate 500 response.

SHALL: All DB operations within a single file upload (insert to `hdd_nodes`, insert to `audit_logs`) SHALL execute within a single MySQL transaction, rolled back atomically on failure.

SHALL: FileService SHALL store a SHA-256 checksum of every uploaded file in the `hdd_nodes` table. On download, the server SHALL recompute the checksum and compare it against the stored value, returning a 409 with an integrity error if they differ.

3.6 Concurrent Write Handling (Gap — New in v2.1)

Gap Description

No file locking or optimistic concurrency strategy is defined. If two users upload a file with the same name to the same folder simultaneously, both may pass the validation step, and the second HDD write will silently overwrite the first — losing a version and potentially creating a corrupt DB state.

Formal Requirements

SHALL: FileService SHALL implement an optimistic lock when creating a new version: before writing to HDD, it SHALL check for an existing record with the same (`dept_id`, `parent_id`, `name`) combination within a database transaction using `SELECT ... FOR UPDATE`.

SHALL: If a concurrent write conflict is detected, the second request SHALL receive a 409 Conflict response instructing the client to retry.

SHOULD: The UI SHOULD display a user-friendly message when a 409 is received on upload, explaining that the file was modified by another user and offering a retry option.

ISTRAC-FMS: Chapter 4 — User Roles & Access Control

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 4: User Roles & Access Control	1
4.1 Role Hierarchy	1
4.2 Permission Matrix	1
4.3 Per-Department Access Levels	2
4.4 Guest User Access (Unauthenticated)	2
4.5 Folder Management Permissions (Gap — New in v2.1)	2
4.6 Admin Runtime Configuration Panel (Gap — New in v2.1)	3

Chapter 4: User Roles & Access Control

4.1 Role Hierarchy

The system has five access levels, each inheriting from the one above it. Every user's permissions are determined by their role plus their per-department access assignment.

L0: Public (unauthenticated user)

Can view landing page, limited file metadata (no downloads)

L1: Super Admin (1 bootstrapped account)

Full system control: departments, users, CMS, HDD sync, audit logs

L2: Department Admin (created by Super Admin)

Manages own department: approve/reject members, upload, delete, view audit log

L3: Department Member (approved by any admin)

Can read + upload within assigned departments (no delete)

L3r: Read-Only Guest (approved with restricted flag)

View + download only, within specifically assigned departments

4.2 Permission Matrix

Action	Public	Guest	Member	Dept Admin	Super Admin
View landing / CMS	Yes	Yes	Yes	Yes	Yes
View file metadata	Limited	Yes	Yes	Yes	Yes
Download files	No	Assigned depts	Own dept	Own dept	All
Upload files	No	No	Own dept	Own dept	All
Delete files	No	No	No	Own dept	All
Create departments	No	No	No	No	Yes
Approve / reject users	No	No	No	Own dept	All
Edit CMS content	No	No	No	No	Yes
View audit logs	No	No	No	Own dept	All
Manage HDD sync	No	No	No	No	Yes
Force-logout any user	No	No	No	No	Yes
Create subfolders	No	No	Own dept (configurable)	Own dept	All
Manage runtime config	No	No	No	No	Yes

4.3 Per-Department Access Levels

A single user can be assigned to multiple departments, each with an independent access level. This supports cross-department roles (e.g., a user who can upload to SIMS but only read from FDD).

Access Level	Permissions Granted
READ_ONLY	View file metadata, download files. Cannot upload, modify, or delete.
READ_WRITE	All READ_ONLY permissions plus upload new files and update file metadata.
READ_WRITE_DELETE	All READ_WRITE permissions plus move files to trash (soft delete).

Additionally, file-level permission overrides can be granted by admins for specific files.

4.4 Guest User Access (Unauthenticated)

Unauthenticated visitors can access the landing page and browse limited file metadata (filename, department, type, size) that the Super Admin has configured as public-visible. They cannot download, upload, or view file contents.

4.5 Folder Management Permissions (Gap — New in v2.1)

Gap Description

The v2.0 document describes department-level HDD paths and the `hdd_nodes` table supports a folder tree via `parent_id`, but the UI and permission rules for user-level folder creation are not specified anywhere. Without this, the folder structure either cannot be managed by users (limiting usability) or is uncontrolled (creating a messy file tree).

Formal Requirements

SHALL: The system SHALL allow users with `READ_WRITE` or higher access to a department to create subfolders within their department's root path, subject to a Super Admin configuration flag (`allow_user_folder_creation`) that can be toggled per department.

SHALL: Only users with `READ_WRITE_DELETE` access or higher SHALL be permitted to rename or delete folders. Deleting a folder SHALL soft-delete all files within it recursively, subject to the same 30-day recovery window.

SHALL: Folder depth SHALL be configurable by the Super Admin (default: maximum 5 levels deep) to prevent unmanageable tree depth.

SHALL: The RBAC middleware SHALL enforce folder-level access checks: a user with `READ_ONLY` access to a department can browse all folders but cannot create, rename, or delete any folder node.

SHOULD: The UI SHOULD display a visual breadcrumb trail reflecting the current folder path, and SHOULD show a visual indicator when a folder is at the maximum allowed depth.

4.6 Admin Runtime Configuration Panel (Gap — New in v2.1)

Gap Description

Settings such as max file upload size, allowed file extensions, virus scan toggle, guest access expiry defaults, and the new folder creation flag all reside in the `.env` file, requiring a server restart to change. There is no admin UI for runtime configuration changes.

Formal Requirements

SHALL: The system SHALL provide a `/admin/settings` page accessible only to Super Admin, allowing runtime modification of the following settings without a server restart:

- Maximum file upload size (bytes)
- Allowed file extension allowlist
- Virus scan enabled/disabled toggle
- Default guest access expiry duration
- Allow user folder creation (per-department toggle)
- HDD sync interval (minutes)
- Download rate limit threshold (downloads per user per hour)

SHALL: All runtime configuration changes SHALL be stored in a `system_config` database table (key-value pairs) and loaded by the application at runtime via a config service that checks the DB on each request (with a short in-memory TTL cache of 60 seconds).

SHALL: Every runtime configuration change SHALL be logged to the `audit_logs` table with the admin's identity, the key changed, the old value, and the new value.

SHALL: Environment variables in `.env` SHALL serve only as bootstrap defaults. If a setting exists in the `system_config` table, the DB value SHALL take precedence over the `.env` value.

ISTRAC-FMS: Chapter 5 — Authentication & User Lifecycle

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 5: Authentication & User Lifecycle	1
5.1 Registration & Approval Flow	1
5.2 JWT Authentication Strategy	2
5.3 Session Security	2
5.4 Email Notifications (Campus SMTP)	2
5.5 Password Reset Flow (Gap — New in v2.1)	2
5.6 Password Policy (Gap — New in v2.1)	3
5.7 Multi-Factor Authentication (Gap — New in v2.1)	3

Chapter 5: Authentication & User Lifecycle

5.1 Registration & Approval Flow

Users cannot self-activate. Every new account goes through an admin approval workflow.

Step 1: User visits /register and fills form

Fields: name, email (campus), employee_id, dept_preference, reason_for_access

Step 2: Account created with status = PENDING

Admin receives in-app notification + campus email

Step 3: Admin reviews the request in /admin/users/pending

APPROVE: System generates 8-character temporary password

Email sent: 'Your account is approved. Temp password: XXXXXXXX'

User status → ACTIVE

REJECT: Rejection reason (optional note) emailed to user

User status → REJECTED

Step 4: User logs in with temp password → forced password change screen

After password change → access granted, temp_pass flag = false

5.2 JWT Authentication Strategy

The system uses a dual-token strategy: a short-lived access token for API requests and a long-lived refresh token stored securely.

Token	Details
Access Token	15-minute TTL. Stored in React memory (not localStorage). Sent in Authorization header.
Refresh Token	7-day TTL. Stored in httpOnly cookie (not accessible to JavaScript). Rotates on every use.
Token Blacklist	Stored in DB table. Checked on every refresh. Invalidated on logout or admin force-logout.
Password Change	All existing tokens invalidated immediately — all sessions logged out.

5.3 Session Security

- Admin can force-logout any user from the admin panel — instantly invalidates their refresh token.
- 5 consecutive failed login attempts trigger a 15-minute account lockout.
- All password changes automatically invalidate all active sessions.
- Guest account access expiry: admin can set an `expires_at` date on department access grants.

5.4 Email Notifications (Campus SMTP)

The application uses Nodemailer connected to the campus internal SMTP server. No external mail service is used.

Emails sent by the system:

- Account approved: includes temporary password.
- Account rejected: includes reason from admin.
- Account suspended: notification to user.
- File uploaded in your department (configurable opt-in).
- Guest access expiry reminder (7 days and 1 day before).

5.5 Password Reset Flow (Gap — New in v2.1)

Gap Description

The v2.0 document defines the registration flow and first-login forced password change in detail, but provides no mechanism for a user who has forgotten their password to regain access. The only implied path is contacting an admin, which is not specified and creates a support burden.

Formal Requirements

SHALL: The system SHALL provide a “Forgot Password” link on the login page.

SHALL: Clicking “Forgot Password” SHALL trigger a form where the user enters their campus email address. The system SHALL send a one-time password reset link to that address via campus SMTP if the email corresponds to an ACTIVE account.

SHALL: The reset token SHALL be a cryptographically random 64-character string, stored as a hash in the database, valid for 30 minutes, and single-use (invalidated immediately after use).

SHALL: The reset link SHALL direct the user to a page where they set a new password. On successful submission, all existing refresh tokens for that user SHALL be invalidated.

SHALL: Password reset requests for non-existent or non-ACTIVE email addresses SHALL return the same success message as valid requests (“If this email exists, a reset link was sent”), preventing user enumeration.

SHALL: Each password reset request SHALL be logged to `audit_logs` with the user ID and IP address.

5.6 Password Policy (Gap — New in v2.1)

Gap Description

The v2.0 document specifies bcrypt cost factor 12 for hashing, but does not define any password complexity requirements, expiry policy, or history enforcement. Without a policy, users may set trivially weak passwords.

Formal Requirements

SHALL: Passwords SHALL meet the following minimum requirements at the time of creation or change:

- Minimum 10 characters in length.
- At least one uppercase letter.
- At least one lowercase letter.
- At least one digit.
- At least one special character (!@#\$%^&*()-_+=).

SHALL: The system SHALL reject passwords that match any of the user’s last 5 passwords (stored as bcrypt hashes in a `password_history` table).

SHOULD: Passwords SHOULD be validated client-side with a live strength indicator before submission, to reduce round-trips for policy failures.

SHALL: Password expiry is NOT enforced automatically in v1.0 (intranet users tend to be stable). This decision SHALL be revisited in v1.1 based on campus IT policy.

5.7 Multi-Factor Authentication (Gap — New in v2.1)

Gap Description

The v2.0 document has no MFA mechanism. On a private intranet this is a lower priority than on a public-facing system, but Super Admin accounts have unrestricted system access and represent a high-value compromise target.

Formal Requirements

SHOULD: The system SHOULD support TOTP-based MFA (RFC 6238, compatible with Google Authenticator or similar apps) as an optional feature for all users.

SHALL: MFA SHALL be mandatory for Super Admin accounts. On first login after the feature is deployed, Super Admin SHALL be forced to enroll a TOTP device before access is granted.

SHALL: MFA enrollment (QR code generation and backup codes) SHALL be accessible via the user's profile settings page.

SHALL: Backup recovery codes (8 single-use codes) SHALL be generated at TOTP enrollment. Using a recovery code SHALL be logged to `audit_logs` and SHALL prompt the user to re-enroll a new TOTP device.

ISTRAC-FMS: Chapter 6 — Functional Requirements

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 6: Functional Requirements	1
6.1 File Management	1
6.2 HDD Sync Daemon	2
6.3 Search	3
6.4 CMS (Admin-Editable Landing Page)	3
6.5 Notifications	3
6.6 Bulk Operations (Gap — New in v2.1)	3
6.7 Intranet File Sharing Links (Gap — New in v2.1)	4

Chapter 6: Functional Requirements

6.1 File Management

6.1.1 Upload

Any user with `READ_WRITE` or higher access to a department can upload files.

1. Client selects file → pre-flight validation (size limit, extension allowlist)
2. Multipart POST to `/api/v1/files/upload`
3. Server: magic byte validation (actual file type, not just extension)
4. Server: SHA-256 checksum computed and stored
5. Server: write bytes to HDD at correct department path
6. DB: insert `hdd_nodes` record with metadata (name, size, mime, path, uploader, checksum)
7. Audit: log `FILE_UPLOAD` event
8. WebSocket: notify all dept members of new file in real-time

6.1.2 Chunked / Resumable Upload (Gap — New in v2.1)

Gap Description The v2.0 document sets a default max upload size of 500 MB but defines only a single-shot multipart POST for uploads. On a campus intranet connection, a 500 MB upload can fail mid-transfer with no recovery path. There is no progress reporting, no resume capability, and a failed upload silently loses all transferred data.

Formal Requirements **SHALL:** The upload system SHALL support chunked upload for files larger than 10 MB (configurable threshold).

SHALL: The client SHALL split large files into chunks of configurable size (default: 5 MB) and upload them sequentially via `POST /api/v1/files/upload/chunk`.

SHALL: The server SHALL maintain an upload session record in the DB, tracking received chunks. An incomplete upload session SHALL expire and be cleaned up after 24 hours.

SHALL: If a chunk upload fails, the client SHALL be able to resume the upload from the last successfully received chunk without re-uploading earlier chunks.

SHALL: On completion of all chunks, the server SHALL assemble the file on HDD, compute the full SHA-256 checksum, validate it against the client-provided checksum, and commit the final `hdd_nodes` record atomically.

SHOULD: The UI SHOULD display a real-time progress bar during chunked uploads, showing percentage complete and current transfer speed.

6.1.3 Download & Preview

- Server streams file bytes from HDD — never loads entire file into memory.
- Preview supported for: PDF (PDF.js), images (native img tag), text/CSV/JSON (syntax-highlighted), video (HTML5 video).
- Office files (docx, xlsx, pptx) show metadata and offer download prompt.
- Every download is logged to `audit_logs` (including download count increment).

6.1.4 Delete (Soft Delete)

- Dept Admin or Super Admin only. Files are never permanently deleted from the UI.
- Soft delete: DB marks `deleted_at = NOW()`. Physical file moved to `/_trash/{dept}/` on HDD.
- Files are recoverable within 30 days (configurable). After that, admin can purge permanently.
- Deletion is audit-logged with the admin's identity.

6.1.5 Version Control

- Uploading a file with the same name to the same folder creates a new version automatically.
- Previous version moved to `/_versions/{file_id}/{version_num}/` on HDD.
- DB tracks full version chain in `file_versions` table.
- UI shows version history panel; any version can be downloaded.

6.2 HDD Sync Daemon

A background daemon reconciles the physical HDD with the database every 15 minutes. It can also be triggered manually by the Super Admin.

`SyncDaemon.run()`:

For each file/folder in HDD tree:

```
if NOT in DB      → insert as UNREGISTERED (admin must review)
if IN DB and HDD → update last_seen, size, modified_at
if IN DB, not HDD → mark status = ORPHANED (alert admin)
```

On completion → emit `sync_completed` event via WebSocket

6.3 Search

- Full-text search on: filename, description, tags, department name.
- MySQL FULLTEXT indexes on `hdd_nodes(name, description, tags)`.
- Results automatically scoped to user's accessible departments.
- Filter operators: `dept:SIMS`, `type:pdf`, `before:2025-01-01`, `after:2024-01-01`.
- Search history per user (last 20 queries).
- All search results SHALL be paginated (see Chapter 9 for pagination requirements).

6.4 CMS (Admin-Editable Landing Page)

CMS Block Key	Purpose
<code>hero_title</code>	Main headline on the landing page
<code>hero_subtitle</code>	Subheadline / tagline
<code>hero_cta_text</code>	Call-to-action button label
<code>featured_doc_ids</code>	File IDs to feature prominently on the homepage
<code>announcements</code>	HTML announcement banner (shown/hidden by admin)
<code>contact_info</code>	Department contact details
<code>org_overview</code>	Organizational overview text or HTML

Changes take effect immediately via WebSocket push — the React client re-fetches updated CMS content with no page reload required.

6.5 Notifications

Real-time notifications are delivered via WebSocket and persisted in the DB for offline users.

- New file uploaded in a department you are a member of.
- Your registration request was approved or rejected.
- A file you favorited has been updated.
- HDD sync completed (admins only).
- Your guest access is expiring in 7 days (then 1 day).

6.6 Bulk Operations (Gap — New in v2.1)

Gap Description

All file operations in v2.0 are one-at-a-time only. This makes bulk administration tasks (e.g., tagging or deleting a batch of files after a project closes) extremely tedious.

Formal Requirements

SHALL: The FileGrid and FileList components SHALL support multi-select via checkboxes.

SHALL: The following bulk API endpoints SHALL be implemented:

- `POST /api/v1/files/bulk-delete` — soft-delete multiple files (admin only)
- `POST /api/v1/files/bulk-tag` — add or remove tags from multiple files
- `POST /api/v1/files/bulk-move` — move multiple files to a different folder within the same department

SHALL: Each bulk operation SHALL be transactional: either all items succeed or the entire batch is rolled back, with an error response listing which items failed.

SHALL: Each individual file action within a bulk operation SHALL be logged separately to `audit_logs`.

SHALL: Bulk operations SHALL be subject to a configurable maximum batch size (default: 100 files per request) to prevent timeouts.

6.7 Intranet File Sharing Links (Gap — New in v2.1)

Gap Description

Currently, to share a file, users must tell colleagues the file name and department. There is no shareable link mechanism.

Formal Requirements

SHALL: Users with at least `READ_ONLY` access to a file SHALL be able to generate a shareable intranet link via `POST /api/v1/files/:id/share`.

SHALL: Share links SHALL be of the form `/share/{token}` where `{token}` is a 32-character cryptographically random string.

SHALL: Each share link SHALL have a configurable expiry (options: 1 hour, 24 hours, 7 days), after which the link SHALL return 410 Gone.

SHALL: Accessing a share link SHALL require the recipient to be authenticated (logged in). Unauthenticated users clicking the link SHALL be redirected to login, then returned to the share URL.

SHALL: Share link usage SHALL be logged to `audit_logs` with the accessor's identity and timestamp.

SHOULD: Share links SHOULD be optionally single-use: a flag at creation time marks the token as consumed after the first successful access.

ISTRAC-FMS: Chapter 7 — Non-Functional Requirements

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 7: Non-Functional Requirements	1
7.1 Performance Targets	1
7.2 Security Requirements	1
7.3 Reliability	3
7.4 Maintainability	3
7.5 Observability & Monitoring (Gap — New in v2.1)	3

Chapter 7: Non-Functional Requirements

7.1 Performance Targets

Metric	Target
Page load (LAN, 100 Mbps)	< 2 seconds
File list render (up to 10,000 files)	< 500 ms (virtualized list with <code>react-virtual</code>)
Search response time	< 300 ms for typical queries
File upload max size	Configurable, default 500 MB per file
Concurrent users supported	200 (intranet, not internet scale)
HDD sync interval	Every 15 minutes (or manual trigger)
Chunked upload chunk size	Default 5 MB per chunk
API list endpoints	All paginated, max 100 items per page

7.2 Security Requirements

- All traffic over HTTPS — self-signed certificate is acceptable on a private intranet.
- HSTS header sent with all HTTPS responses (`Strict-Transport-Security: max-age=31536000`).
- Passwords hashed with bcrypt, cost factor 12.
- No file paths ever exposed in API responses — only file IDs. Server resolves path internally.
- Rate limiting: 5 failed logins → 15-minute lockout per IP.
- Path traversal prevention: all client-provided paths are sanitized server-side.
- File type validation: check actual magic bytes (file signature), not just filename extension.
- Upload virus scan hook: architecture includes a pluggable scan step (ClamAV or similar).

- RBAC middleware enforced on every API route.

7.2.1 HTTP Security Headers (Gap — New in v2.1)

Gap Description The v2.0 document mentions HTTPS and HSTS but does not define the full set of HTTP security response headers. Without these, the application is vulnerable to clickjacking, MIME sniffing, and content injection attacks even on a private intranet.

Formal Requirements **SHALL:** The following HTTP response headers SHALL be sent on all responses from Nginx:

- Strict-Transport-Security: max-age=31536000; includeSubDomains
- X-Frame-Options: DENY
- X-Content-Type-Options: nosniff
- Referrer-Policy: strict-origin-when-cross-origin
- Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; connect-src 'self' wss:

SHALL: The Server response header SHALL be removed or obscured by Nginx to avoid fingerprinting.

7.2.2 CSRF Protection (Gap — New in v2.1)

Gap Description The v2.0 document does not define any CSRF protection strategy. While JWT in the Authorization header (not a cookie) provides some CSRF protection for API calls, the refresh token is stored in an httpOnly cookie, making the `/api/v1/auth/refresh` endpoint potentially vulnerable to CSRF attacks that force a token rotation.

Formal Requirements **SHALL:** The refresh token cookie SHALL be set with `SameSite=Strict` attribute, which prevents the cookie from being sent in cross-site requests.

SHALL: All state-mutating API endpoints that accept the httpOnly cookie (specifically `/api/v1/auth/refresh` and `/api/v1/auth/logout`) SHALL additionally validate a CSRF token sent in a custom request header (`X-CSRF-Token`), generated at login and stored in React memory.

SHALL: The CSRF token SHALL be rotated on every successful token refresh.

7.2.3 CORS Policy (Gap — New in v2.1)

Gap Description No CORS configuration is defined. Since the React SPA and API are served from the same Nginx origin, a restrictive CORS policy is appropriate and safe.

Formal Requirements **SHALL:** The Express server SHALL configure CORS to allow requests only from the application's own origin (the intranet hostname/IP).

SHALL: The CORS policy SHALL NOT use wildcard origins (*). The allowed origin SHALL be set from the `APP_ORIGIN` environment variable.

SHALL: The CORS configuration SHALL explicitly list allowed methods (GET, POST, PUT, DELETE, PATCH) and allowed headers (`Authorization`, `Content-Type`, `X-CSRF-Token`).

7.2.4 File Encryption at Rest (Gap — New in v2.1)

Gap Description Files on the HDD are stored in plaintext. An attacker with OS-level or physical access to the drive can read all files without any application authentication. For a system managing potentially sensitive documents, this is a risk.

Formal Requirements **SHOULD:** The system SHOULD support optional AES-256-GCM encryption at rest for designated departments, configurable by Super Admin.

SHALL: If encryption is enabled for a department, the HddService SHALL encrypt file bytes before writing to disk and decrypt on streaming read. The encryption key per department SHALL be stored in a secrets vault (initially a dedicated, access-restricted DB table; upgradeable to HashiCorp Vault in future phases).

SHALL: Encryption keys SHALL never be stored alongside the files they protect. The key derivation SHALL use a master secret from `.env` combined with a per-department salt stored in the DB.

7.3 Reliability

- MySQL daily automated backup via `mysqldump` cron job — 30-day retention.
- HDD failure detection: sync daemon alerts admin if the mount point becomes unavailable.
- Graceful degradation: if HDD is offline, application loads from cached DB metadata and shows ‘File Unavailable’ indicator.
- PM2 process management: Node.js process auto-restarts on crash.

7.4 Maintainability

- TypeScript strict mode enforced across frontend and backend.
- ESLint + Prettier with pre-commit hooks.
- All API routes documented in OpenAPI 3.0 (Swagger UI available on intranet).
- Database schema changes managed via migration files (db-migrate or Flyway).
- Environment-based config — all paths, secrets, and settings in `.env` (never hardcoded).

7.5 Observability & Monitoring (Gap — New in v2.1)

Gap Description

The v2.0 document covers audit logging for compliance but does not address application-level error logging, performance monitoring, or uptime alerting. Without observability tooling, debugging production issues requires SSHing into the server and reading raw logs.

Formal Requirements

SHALL: All Node.js application logs SHALL be written in structured JSON format using a logging library (e.g., `pino`) with fields: `timestamp`, `level`, `requestId`, `userId`, `method`, `path`, `statusCode`, `durationMs`, `error`.

SHALL: Log files SHALL be managed by PM2’s built-in log rotation with a maximum file size of 100 MB and a retention of 30 days.

SHALL: Unhandled exceptions and unhandled promise rejections SHALL be caught globally, logged at **error** level, and trigger a PM2 restart.

SHALL: A `/api/v1/health` endpoint SHALL be available (no auth required) returning 200 OK with a JSON body indicating: application status, DB connection status, HDD mount status, and current timestamp. This endpoint SHALL be used for uptime monitoring.

SHOULD: An internal uptime monitoring check (e.g., a cron-driven curl to `/api/v1/health`) SHOULD be configured on the RHEL server to send an alert email to the Super Admin if the health check fails for 3 consecutive checks.

ISTRAC-FMS: Chapter 8 — Database Schema Design

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 8: Database Schema Design	1
8.1 Entity Relationship Overview	1
8.2 Key Schema Details	2
8.3 Tag Normalization Migration Note (Gap — New in v2.1)	4
8.4 Database Indexes	4

Chapter 8: Database Schema Design

8.1 Entity Relationship Overview

The database contains 12 core tables (10 original + 2 new in v2.1). The `hdd_nodes` table is the central entity.

Table	Purpose	Key Relationships
<code>users</code>	All user accounts with role and status	Referenced by almost every table
<code>departments</code>	Department registry with HDD path mapping	Has many users via <code>user_department_access</code>
<code>user_department_access</code>	Per-user, per-department access level	Links users to departments
<code>hdd_nodes</code>	Mirrors HDD file/folder tree with metadata	Central entity — files, versions, perms
<code>file_versions</code>	Version history for files	Many-to-one with <code>hdd_nodes</code>
<code>file_permissions</code>	File-level permission overrides	Links users to specific files
<code>audit_logs</code>	Immutable append-only event log	References users and resources
<code>cms_blocks</code>	Admin-editable landing page content	Standalone
<code>notifications</code>	User notification inbox	References users
<code>refresh_tokens</code>	Active JWT refresh tokens	References users
<code>password_history</code>	Last 5 password hashes per user (new v2.1)	References users

Table	Purpose	Key Relationships
system_config	Runtime configuration key-value store (new v2.1)	Standalone
tags	Normalized tag registry (new v2.1)	Many-to-many with hdd_nodes via hdd_node_tags
hdd_node_tags	Junction table for file-tag relationships (new v2.1)	Links hdd_nodes to tags

8.2 Key Schema Details

Users Table

users:

```

id          CHAR(36)      PK DEFAULT (UUID())
name        VARCHAR(255) NOT NULL
email       VARCHAR(255) UNIQUE NOT NULL  -- campus email
employee_id VARCHAR(50)   UNIQUE
password    VARCHAR(255) NOT NULL         -- bcrypt hash
role        ENUM('SUPER_ADMIN', 'DEPT_ADMIN', 'MEMBER', 'GUEST')
status      ENUM('PENDING', 'ACTIVE', 'SUSPENDED', 'REJECTED')
temp_pass   BOOLEAN      DEFAULT TRUE     -- force password change on first login
mfa_secret  VARCHAR(255) NULL             -- TOTP secret (NEW v2.1, encrypted at rest)
mfa_enabled BOOLEAN      DEFAULT FALSE   -- NEW v2.1
last_login  DATETIME
created_at  DATETIME      DEFAULT NOW()

```

hdd_nodes Table (Core File Record)

hdd_nodes:

```

id          CHAR(36)      PK DEFAULT (UUID())
dept_id     CHAR(36)      FK → departments(id)
parent_id   CHAR(36)      FK → hdd_nodes(id)  -- folder tree structure
node_type   ENUM('FOLDER', 'FILE')
name        VARCHAR(500) NOT NULL
hdd_path    VARCHAR(1000) UNIQUE             -- absolute path on server
size_bytes  BIGINT
mime_type   VARCHAR(100)
extension   VARCHAR(20)
sha256      CHAR(64)      NULL               -- file integrity checksum (NEW v2.1)
status      ENUM('ACTIVE', 'ORPHANED', 'DELETED', 'UNREGISTERED')
description TEXT
version_count INT          DEFAULT 1
last_synced DATETIME
deleted_at  DATETIME      NULL               -- soft delete timestamp
FULLTEXT INDEX ft_search (name, description)

```

tags Table (New in v2.1 — Normalized)

```
tags:
  id          CHAR(36)      PK DEFAULT (UUID())
  name        VARCHAR(100) UNIQUE NOT NULL
  created_by  CHAR(36)      FK → users(id)
  created_at  DATETIME      DEFAULT NOW()
```

```
hdd_node_tags:
  node_id     CHAR(36) FK → hdd_nodes(id)
  tag_id      CHAR(36) FK → tags(id)
  PRIMARY KEY (node_id, tag_id)
```

Note: Tags are no longer stored as comma-separated VARCHAR on hdd_nodes. The tags VARCHAR column is removed in v2.1 and replaced with this normalized structure. FULLTEXT search on tags is supported by a denormalized tag_string computed column updated by trigger.

audit_logs Table (Append-Only)

```
audit_logs:
  id          BIGINT        AUTO_INCREMENT PK
  user_id     CHAR(36)      FK → users(id)
  action       VARCHAR(100) -- e.g. FILE_DOWNLOAD, USER_APPROVED
  resource_type VARCHAR(50) -- 'file', 'user', 'department'
  resource_id  CHAR(36)
  old_value   JSON          -- state before change
  new_value   JSON          -- state after change
  ip_address   VARCHAR(45)
  user_agent   VARCHAR(500)
  created_at  DATETIME      DEFAULT NOW()
```

-- !! No UPDATE or DELETE permissions on this table in production !!

password_history Table (New in v2.1)

```
password_history:
  id          BIGINT        AUTO_INCREMENT PK
  user_id     CHAR(36)      FK → users(id)
  password    VARCHAR(255) NOT NULL -- bcrypt hash
  created_at  DATETIME      DEFAULT NOW()
```

-- Retain only last 5 records per user (enforced at application layer)

system_config Table (New in v2.1)

```
system_config:
  config_key  VARCHAR(100) PK
  config_value TEXT      NOT NULL
```

updated_by	CHAR(36)	FK → users(id)
updated_at	DATETIME	DEFAULT NOW()

8.3 Tag Normalization Migration Note (Gap — New in v2.1)

Gap Description

In v2.0, tags were stored as a comma-separated VARCHAR(500) column on `hdd_nodes`. This approach works for basic FULLTEXT search but makes tag-based filtering, tag rename, and tag autocomplete complex and fragile.

Formal Requirements

SHALL: The v2.1 schema SHALL replace the `tags` VARCHAR(500) column on `hdd_nodes` with a normalized `tags` table and `hdd_node_tags` junction table as defined above.

SHALL: A migration script SHALL be provided that reads existing comma-separated tag strings, creates the normalized tag records, and populates `hdd_node_tags` without data loss.

SHALL: The system SHALL provide admin-facing tag management: list all tags, rename a tag (propagates to all files), and merge two tags into one.

SHALL: Tag autocomplete SHALL be supported on the file upload and file edit forms, driven by a GET `/api/v1/tags?q=` endpoint returning matching tag names.

8.4 Database Indexes

The following indexes SHALL exist beyond the primary keys:

```
-- Performance-critical lookups
CREATE INDEX idx_hdd_nodes_dept      ON hdd_nodes(dept_id, status);
CREATE INDEX idx_hdd_nodes_parent    ON hdd_nodes(parent_id);
CREATE INDEX idx_audit_logs_user     ON audit_logs(user_id, created_at);
CREATE INDEX idx_audit_logs_action   ON audit_logs(action, created_at);
CREATE INDEX idx_notifications_user  ON notifications(user_id, read_at);
CREATE INDEX idx_refresh_tokens_usr  ON refresh_tokens(user_id);
```

ISTRAC-FMS: Chapter 9 — API Contract Design

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 9: API Contract Design	1
9.1 API Design Principles	1
9.2 Route Summary	1
9.3 Pagination Strategy (Gap — New in v2.1)	3
9.4 Standard Error Response Format	4

Chapter 9: API Contract Design

9.1 API Design Principles

- All routes prefixed with `/api/v1/` — versioning from day one allows future breaking changes.
- JSON request and response bodies throughout.
- HTTP status codes used correctly: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 410 Gone, 500 Internal Server Error.
- Every route documented in OpenAPI 3.0 (Swagger UI served at `/api/docs` on intranet).
- All list endpoints return paginated responses (see 9.3).

9.2 Route Summary

Method + Path	Auth	Role Gate	Description
POST <code>/api/v1/auth/register</code>	No	Public	Submit registration request
POST <code>/api/v1/auth/login</code>	No	Public	Login, receive tokens
POST <code>/api/v1/auth/refresh</code>	Cookie	Public	Rotate refresh token
POST <code>/api/v1/auth/logout</code>	JWT	Authenticated	Invalidate refresh token
POST <code>/api/v1/auth/change-password</code>	JWT	Authenticated	Change own password
POST <code>/api/v1/auth/forgot-password</code>	No	Public	Initiate password reset

Method + Path	Auth	Role Gate	Description
POST /api/v1/auth/reset-password	No	Public	Complete password reset with token
POST /api/v1/auth/mfa/enroll	JWT	Authenticated	Enroll TOTP device
POST /api/v1/auth/mfa/verify	JWT	Authenticated	Verify TOTP code during login
GET /api/v1/users	JWT	SUPER_ADMIN	List all users (paginated)
GET /api/v1/users/pending	JWT	SUPER_ADMIN DEPT_ADMIN	List pending registrations
POST /api/v1/users/:id/approve	JWT	SUPER_ADMIN DEPT_ADMIN	Approve registration
POST /api/v1/users/:id/reject	JWT	SUPER_ADMIN DEPT_ADMIN	Reject with reason
POST /api/v1/users/:id/force-logout	JWT	SUPER_ADMIN	Invalidate user session
GET /api/v1/departments	JWT	Authenticated	List departments (scoped)
POST /api/v1/departments	JWT	SUPER_ADMIN	Create department
POST /api/v1/departments/:id/members	JWT	SUPER_ADMIN DEPT_ADMIN	Add user to department
GET /api/v1/files/:id/download	JWT	Has access	Stream file bytes from HDD
POST /api/v1/files/upload	JWT	WRITE access	Upload file (single-shot, 10 MB)
POST /api/v1/files/upload/chunk	JWT	WRITE access	Upload a single chunk
POST /api/v1/files/upload/complete	JWT	WRITE access	Finalize chunked upload
DELETE /api/v1/files/:id	JWT	DEPT_ADMIN SUPER_ADMIN	Soft delete file
GET /api/v1/files/:id/versions	JWT	Has access	List version history
POST /api/v1/files/:id/share	JWT	Has access	Generate share link
GET /api/v1/share/:token	JWT	Authenticated	Access shared file
POST /api/v1/files/bulk-delete	JWT	DEPT_ADMIN SUPER_ADMIN	Bulk soft delete
POST /api/v1/files/bulk-tag	JWT	WRITE access	Bulk tag files
POST /api/v1/files/bulk-move	JWT	WRITE access	Bulk move files
GET /api/v1/browse/:deptId/*path	JWT	Authenticated	Navigate folder tree

Method + Path	Auth	Role Gate	Description
POST /api/v1/browse/:deptId/folder	JWT	WRITE access	Create subfolder
GET /api/v1/search?q=&dept=&type=	JWT	Authenticated	Full-text search (paginated)
GET /api/v1/tags?q=	JWT	Authenticated	Tag autocomplete
GET /api/v1/admin/stats	JWT	SUPER_ADMIN	System statistics
GET /api/v1/admin/audit-logs	JWT	SUPER_ADMIN DEPT_ADMIN	View audit log (paginated)
POST /api/v1/admin/sync/trigger	JWT	SUPER_ADMIN	Manually trigger HDD sync
GET /api/v1/admin/settings	JWT	SUPER_ADMIN	Get runtime config
PUT /api/v1/admin/settings/:key	JWT	SUPER_ADMIN	Update runtime config value
GET /api/v1/cms	No	Public	Get all CMS content blocks
PUT /api/v1/cms/:blockKey	JWT	SUPER_ADMIN	Update CMS block
GET /api/v1/health	No	Public	Application health check
GET /api/v1/health/hdd	JWT	SUPER_ADMIN	HDD mount health status

9.3 Pagination Strategy (Gap — New in v2.1)

Gap Description

No pagination strategy is defined in v2.0. With potentially 10,000+ files in a department and growing audit logs, returning unpaginated results from list endpoints will cause serious memory and performance issues as data grows.

Formal Requirements

SHALL: All list endpoints (files, users, audit logs, search results, notifications, versions) SHALL support pagination.

SHALL: The system SHALL use **cursor-based pagination** (not offset-based) for `audit_logs` and `notifications`, as these are append-only tables where rows are never deleted. Cursor-based pagination is stable and performant on large tables.

SHALL: The system SHALL use **offset-based pagination** for file lists and user lists (smaller datasets, random access needed for UI). Page size SHALL default to 50 and SHALL be configurable per request up to a maximum of 100.

SHALL: All paginated responses SHALL follow this envelope format:

```

{
  "data": [ ... ],
  "pagination": {
    "total": 1543,
    "page": 2,
    "pageSize": 50,
    "totalPages": 31,
    "hasNext": true,
    "hasPrev": true
  }
}

```

For cursor-based endpoints, `nextCursor` and `prevCursor` replace `page` and `totalPages`.

SHALL: The React frontend SHALL use React Query's `useInfiniteQuery` for scroll-based file lists and `usePaginatedQuery` for table-based views (users, audit logs).

9.4 Standard Error Response Format

All error responses SHALL use this format:

```

{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable message",
    "details": [ { "field": "email", "message": "Invalid format" } ]
  },
  "requestId": "uuid"
}

```

Error codes SHALL be stable string constants, documented in the OpenAPI spec, allowing the frontend to handle specific errors programmatically.

ISTRAC-FMS: Chapter 10 — Security Model

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 10: Security Model	1
10.1 Defense in Depth — Eight Layers	1
10.2 Path Traversal Prevention	2
10.3 Secrets Management	2
10.4 Download Rate Limiting (Gap — New in v2.1)	2
10.5 HDD Mount Health Check (Gap — New in v2.1)	3
10.6 LDAP / Active Directory Integration (Gap — New in v2.1)	3

Chapter 10: Security Model

10.1 Defense in Depth — Eight Layers

Security is implemented at every layer of the stack. An attacker who bypasses one layer should be caught by the next. v2.1 adds an eighth layer for headers/CSRF.

Layer #	Layer Name	Mechanism
1	Network	Private intranet only — physically impossible to reach from internet
2	Transport	HTTPS with HSTS + full security header suite
3	Authentication	JWT (15 min) + rotating httpOnly refresh token (7 days) + optional TOTP MFA
4	Authorization	RBAC middleware on every route + per-file permission check in service layer
5	Input Validation	Zod schema validation on all request bodies — invalid inputs rejected at boundary

Layer #	Layer Name	Mechanism
6	File Safety	Magic byte validation + extension allowlist + SHA-256 integrity + virus scan hook
7	Audit	All mutations logged immutably with user ID + IP + timestamp
8	Browser Defense	CSRF tokens + SameSite=Strict cookies + full HTTP security header suite

10.2 Path Traversal Prevention

File paths are never taken from user input. The client always sends a file ID; the server resolves the path from the database.

```
// SAFE pattern - path always comes from DB, never from user input
const fileRecord = await FileRepo.findById(fileId); // DB lookup
const safePath = path.resolve(MOUNT_ROOT, fileRecord.hdd_path);

// Guard: path must be inside the mount root
if (!safePath.startsWith(MOUNT_ROOT)) {
  throw new ForbiddenError('Path traversal attempt detected');
}

// Only now do we read the file
return fs.createReadStream(safePath);
```

10.3 Secrets Management

- All secrets stored in `.env` file — never committed to source control.
- JWT signing secrets: minimum 256-bit random strings (generated with `openssl rand`).
- Database credentials: dedicated MySQL user with minimum privileges (`SELECT`, `INSERT`, `UPDATE` — no `DROP/CREATE TABLE` in production).
- Email credentials: dedicated campus SMTP service account.
- `.env` file permissions: `chmod 600` on RHEL server — readable only by the service user.

10.4 Download Rate Limiting (Gap — New in v2.1)

Gap Description

v2.0 applies rate limiting only to login attempts (5 failures → 15-minute logout). File downloads have no rate limit. A compromised or malicious account could bulk-download all accessible files without triggering any alert.

Formal Requirements

SHALL: A per-user download rate limit SHALL be enforced: default 100 file downloads per rolling 60-minute window (configurable via admin runtime settings).

SHALL: If a user exceeds the download threshold, subsequent download requests SHALL return 429 Too Many Requests with a `Retry-After` header.

SHALL: A configurable anomaly alert SHALL be sent to the Super Admin via email and in-app notification if a user downloads more than N files within M minutes (default: 50 files in 10 minutes). This threshold is configurable in the admin settings panel.

SHALL: Download rate limit events SHALL be logged to `audit_logs` with action `DOWNLOAD_RATE_LIMIT_EXCEEDED`.

10.5 HDD Mount Health Check (Gap — New in v2.1)

Gap Description

The sync daemon detects HDD unavailability during its 15-minute scan cycles, but this is reactive. A proactive health check is needed so that HDD unavailability is detected immediately, not on the next sync cycle.

Formal Requirements

SHALL: The application SHALL run a background health check on the HDD mount point every 60 seconds (configurable).

SHALL: The health check SHALL verify the mount point is accessible and writable by writing and immediately deleting a small probe file at `{MOUNT_ROOT}/.health_probe`.

SHALL: If the health check fails, the system SHALL:

1. Immediately send an alert email to the Super Admin via campus SMTP.
2. Set a system-wide degraded state flag visible to all logged-in users as a banner: “File storage is currently unavailable. Downloads and uploads are disabled.”
3. Disable upload and download endpoints (return 503 `Service Unavailable`) until the health check recovers.
4. Log a `HDD_MOUNT_UNAVAILABLE` event to `audit_logs`.

SHALL: When the health check recovers, the system SHALL send a recovery email to the Super Admin and clear the degraded state banner.

SHALL: A `GET /api/v1/health/hdd` endpoint (SUPER_ADMIN only) SHALL expose the current mount status, last check timestamp, and last failure timestamp.

10.6 LDAP / Active Directory Integration (Gap — New in v2.1)

Gap Description

The v2.0 system manages its own user credential store independently of the campus Active Directory / LDAP. This creates a parallel identity system: users have separate passwords for ISTRAC-FMS, admins must manually manage accounts that already exist in AD, and offboarding a user from campus does not automatically revoke ISTRAC access.

Formal Requirements

SHALL: The system SHALL integrate Passport.js LDAP strategy (`passport-ldapauth`) for user authentication. Users SHALL log in with their campus AD credentials.

SHALL: The application SHALL NOT store passwords for LDAP-authenticated users. The `password` column SHALL remain for non-LDAP fallback (Super Admin bootstrapped account only).

SHALL: On successful LDAP authentication, the system SHALL map the LDAP user `uid/sAMAccountName` to a local `users` record by `employee_id`. If no local record exists, the system SHALL create a `PENDING` record and require admin approval before granting access (preserving the existing approval workflow).

SHALL: The LDAP connection settings (server URL, bind DN, bind password, search base) SHALL be stored in `.env` and never hardcoded.

SHALL: If the LDAP server is unreachable, the system SHALL return a clear error to the user and log `LDAP_UNAVAILABLE` to `audit_logs`. The system SHALL NOT fall back to local password auth for LDAP users.

SHOULD: The Super Admin bootstrapped account SHOULD remain as a local password-authenticated fallback to allow recovery if the LDAP server is down.

ISTRAC-FMS: Chapter 11 — Deployment Architecture

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 11: Deployment Architecture (Red Hat Linux)	1
11.1 Server Layout	1
11.2 RHEL-Specific Setup Steps	2
11.3 Nginx Security Header Configuration	2
11.4 Process Management with PM2	3
11.5 Backup Strategy	3
11.6 Disaster Recovery Plan (Gap — New in v2.1)	4

Chapter 11: Deployment Architecture (Red Hat Linux)

11.1 Server Layout

All components run on a single RHEL server (or split to two for DB isolation). Nginx acts as the entry point.

Red Hat Enterprise Linux Server

Nginx (:443 HTTPS)

/ → serve /var/www/istrac-fms/dist (React SPA)
/api/* → proxy to localhost:3000

Node.js Express (:3000)

Managed by PM2 (auto-restart + log rotation)

MySQL (:3306)

Dedicated user: istrac_app (limited privileges)

HDD / NAS mount

/mnt/istrac-data/ (fstab or autofs)

11.2 RHEL-Specific Setup Steps

```
# 1. Install Node.js (via NodeSource RPM)
sudo dnf install nodejs npm

# 2. Install Nginx
sudo dnf install nginx
sudo systemctl enable nginx

# 3. Install MySQL
sudo dnf install mysql-server
sudo systemctl enable mysqld

# 4. Install PM2 globally
npm install -g pm2
pm2 startup systemd    # auto-start on boot

# 5. Configure firewall (allow HTTPS only)
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload

# 6. SELinux policy (allow Nginx to proxy to Node.js)
sudo setsebool -P httpd_can_network_connect 1

# 7. Mount HDD / NAS
# Add to /etc/fstab:
# //server/share /mnt/istrac-data cifs credentials=/etc/samba-creds 0 0
```

11.3 Nginx Security Header Configuration

The following SHALL be added to the Nginx server block (see Chapter 7.2.1):

```
server {
    listen 443 ssl;
    ssl_certificate      /etc/ssl/istrac/cert.pem;
    ssl_certificate_key  /etc/ssl/istrac/key.pem;

    # Security headers
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'";
    server_tokens off;

    location / {
        root /var/www/istrac-fms/dist;
        try_files $uri /index.html;
    }
}
```

```

    location /api/ {
        proxy_pass http://localhost:3000;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

# Redirect HTTP to HTTPS
server {
    listen 80;
    return 301 https://$host$request_uri;
}

```

11.4 Process Management with PM2

```

// ecosystem.config.js
module.exports = {
  apps: [{
    name: 'istrac-fms',
    script: 'dist/server.js',
    instances: 1,           // increase to 'max' for multi-core later
    exec_mode: 'fork',      // change to 'cluster' for horizontal scaling
    env: { NODE_ENV: 'production', PORT: 3000 },
    error_file: './logs/err.log',
    out_file: './logs/out.log',
    log_date_format: 'YYYY-MM-DD HH:mm:ss',
    max_memory_restart: '500M',
  }]
};

pm2 start ecosystem.config.js
pm2 save    # persist process list across reboots

```

11.5 Backup Strategy

Component	Backup Approach
MySQL Database	<code>mysqldump</code> via daily cron job, retained 30 days on separate partition
HDD / NAS	RAID-1 mirroring or nightly <code>rsync</code> to backup drive (OS-level)
<code>.env</code> config	Manually backed up to secure offline storage (not in git)
PM2 ecosystem config	Backed up alongside <code>.env</code>

11.6 Disaster Recovery Plan (Gap — New in v2.1)

Gap Description

v2.0 defines backup mechanisms but provides no documented recovery procedure. In the event of a server failure, the operations team has no step-by-step guide to restore the system, and no RTO/RPO targets are defined.

Formal Requirements

SHALL: The project SHALL document a Disaster Recovery Procedure covering at minimum:

1. Assess failure type (DB corruption, HDD failure, full server loss, partial OS failure).
2. Stop the application: `pm2 stop all`.
3. Restore MySQL from the most recent `mysqldump` backup: `mysql -u root -p istrac_fms < backup.sql`.
4. Verify HDD/NAS mount is intact or restore from RAID/rsync backup.
5. Validate `.env` config is present and correct.
6. Run database migrations: `npm run db:migrate`.
7. Restart application: `pm2 restart all`.
8. Run smoke tests against `/api/v1/health` and a sample file download.
9. Notify Super Admin and affected department admins.

SHALL: The system SHALL define the following recovery targets:

- **RPO (Recovery Point Objective):** Maximum 24 hours of data loss (aligned with daily backup cadence).
- **RTO (Recovery Time Objective):** System SHALL be restorable within 4 hours of a declared failure by a server-familiar engineer following the documented procedure.

SHALL: The Disaster Recovery Procedure document SHALL be stored in the project repository and on secure offline storage alongside the `.env` backup.

SHALL: A quarterly DR drill SHALL be performed: restore a staging environment from a recent backup and validate that the application functions correctly. Results SHALL be documented.

ISTRAC-FMS: Chapter 12 — Scalability & Future Expansion

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 12: Scalability & Future Expansion	1
12.1 Scaling Roadmap	1
12.2 Adding External Storage Systems	2
12.3 What We Are Intentionally NOT Building (Now)	2
12.4 WebSocket Security & Reliability (Gap — New in v2.1)	2

Chapter 12: Scalability & Future Expansion

12.1 Scaling Roadmap

The system is designed to scale gradually without requiring architectural rewrites. Each tier can be scaled independently.

Phase	Trigger	Action
Phase 1 (Now)	Up to 200 users	Single server, single Node.js process, single MySQL instance
Phase 2	200–500 users, slow API	Node.js cluster mode (PM2 instances: max) — uses all CPU cores
Phase 3	DB reads becoming slow	MySQL read replica for audit log / search queries
Phase 4	Session + notification load	Add Redis for token blacklist and notification queues
Phase 5	Multiple server rooms	NFS mount shared across servers, Node.js behind load balancer
NOT planned	Over-engineering	No Kubernetes, no microservices, no cloud migration

12.2 Adding External Storage Systems

The `IStorageProvider` interface makes adding new storage backends possible without touching service logic.

```
// IStorageProvider interface (future-proof)
interface IStorageProvider {
    read(path: string): Promise<ReadableStream>;
    write(path: string, data: Buffer): Promise<void>;
    delete(path: string): Promise<void>;
    list(dirPath: string): Promise<NodeInfo[]>;
    exists(path: string): Promise<boolean>;
}

// Current implementation
class LocalHddProvider implements IStorageProvider { ... }

// Future implementations (plug in without touching service logic)
class SambaShareProvider implements IStorageProvider { ... }
class NfsProvider implements IStorageProvider { ... }
```

To add a new storage system: implement the interface, register it in the DI container, and map department HDD root paths to the new provider.

12.3 What We Are Intentionally NOT Building (Now)

The following are intentionally excluded from v1.0:

- Redis caching layer
- Elasticsearch for advanced search
- Email notification digest batching
- Mobile native app
- SSO / LDAP integration (moved to critical gap — see Ch. 10)
- Full microservices decomposition

12.4 WebSocket Security & Reliability (Gap — New in v2.1)

Gap Description

The WebSocket server is described as a real-time notification mechanism, but v2.0 provides no specification for how the WebSocket connection is authenticated, how it handles reconnections, or how it behaves under load. An unauthenticated WebSocket endpoint on an intranet could allow any connected user to receive notifications for departments they are not a member of.

Formal Requirements

SHALL: WebSocket connections SHALL be authenticated. On connection, the client SHALL send the current JWT access token in the connection URL query parameter (`?token=...`) or as the first message after connection. The server SHALL validate the token and associate the socket with the authenticated user ID.

SHALL: If the JWT is invalid or expired, the server SHALL immediately close the WebSocket connection with code 4401 (Unauthorized).

SHALL: The server SHALL scope all WebSocket broadcasts: a notification for `dept:SIMS` SHALL only be delivered to sockets belonging to users who have access to the SIMS department. Cross-department leakage is prohibited.

SHALL: The client SHALL implement exponential backoff reconnection: on disconnect, attempt reconnect after 1s, 2s, 4s, 8s, 16s, then once per minute. On successful reconnect, the client SHALL re-fetch the notification inbox from the REST API to recover any missed messages.

SHALL: A WebSocket heartbeat (ping/pong) SHALL be implemented with a 30-second interval. Connections that fail to respond to 3 consecutive pings SHALL be terminated and the socket cleaned up from the server's connection map.

SHOULD: The WebSocket server SHOULD maintain a connection count metric exposed via the `/api/v1/health` endpoint.

ISTRAC-FMS: Chapter 13 — Key Design Decisions & Rationale

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 13: Key Design Decisions & Rationale	1
13.1 Original Decisions (v2.0)	1
13.2 New Decisions Added in v2.1	2
13.3 Decisions Explicitly Deferred	3

Chapter 13: Key Design Decisions & Rationale

Every major architectural choice is documented here with the reasoning behind it, so the team understands not just what was decided but why.

13.1 Original Decisions (v2.0)

Decision	Choice Made	Why This Was Chosen
File storage	HDD (not DB BLOBs)	DB BLOBs would bloat MySQL, hurt performance, and complicate backup. Files stay on HDD where they already are.
Authentication	JWT + httpOnly refresh cookie	Stateless access token scales without sticky sessions. HttpOnly cookie prevents XSS token theft.
Database	MySQL 8.0	Team familiarity, FULLTEXT search sufficient at intranet scale, no extra infra needed.
Frontend state	React Query + Zustand	React Query handles server state (caching, background sync). Zustand handles UI state. No Redux boilerplate.

Decision	Choice Made	Why This Was Chosen
Search	MySQL FULLTEXT	Sufficient for intranet file counts. Zero extra infrastructure vs Elasticsearch.
Soft delete	Yes — files moved to <code>_trash/</code>	Compliance requirement. Enables 30-day recovery. Admins can permanently purge after review.
Audit log	Immutable append-only DB table	No UPDATE or DELETE on <code>audit_logs</code> ever. Compliance record. Exportable as CSV.
Real-time	WebSocket (<code>ws</code> library)	Native Node.js, no extra broker needed. Sufficient for intranet scale.
Email	Nodemailer + campus SMTP	Intranet-only system; external email services are inappropriate. Campus SMTP keeps mail internal.
OS	Red Hat Enterprise Linux	Organization standard. RHEL has long-term stability, enterprise support, and SELinux.
Reverse proxy	Nginx	Handles HTTPS termination, static file serving, and API proxying. Faster than Node.js for static assets.

13.2 New Decisions Added in v2.1

Decision	Choice Made	Why This Was Chosen
Password reset	Time-limited token via campus email	Keeps recovery in-band with existing SMTP infrastructure. No SMS required.
CSRF defense	SameSite=Strict + CSRF token in header	SameSite=Strict handles the cookie case; custom header CSRF token handles the refresh endpoint. Minimal complexity.
Pagination	Cursor-based for logs, offset for files	Audit logs are append-only and large — cursor pagination is stable. File lists need random page access for UI.

Decision	Choice Made	Why This Was Chosen
Tag storage	Normalized tables (tags + junction)	Enables rename-across-all-files, proper autocomplete, and merge operations. Slight complexity increase is justified.
Chunked upload	Client-side chunking (5 MB) + server assembly	Enables resumable uploads without a specialist server. Compatible with existing HDD write abstraction.
MFA	TOTP (RFC 6238)	Standard, well-supported, no external service required. Works with campus air-gapped intranet.
Logging	Structured JSON (pino)	Machine-parseable, low overhead, compatible with future log aggregation tools.
LDAP integration	Passport.js LDAP strategy	Integrates with existing campus AD without duplicating identity management. No password storage for LDAP users.
DR targets	RPO 24h / RTO 4h	Aligned with daily backup cadence. 4h RTO is achievable with documented procedure without 24/7 on-call.

13.3 Decisions Explicitly Deferred

Decision	Status	Reason
Redis for token blacklist	Phase 4	Current DB table approach sufficient at 200 users. Add when refresh query latency is measurable.
Elasticsearch	Not planned for v1	MySQL FULLTEXT sufficient. Elasticsearch adds operational overhead not justified at this scale.
File encryption at rest	Optional / department-level	Full encryption adds key management complexity. Offered as opt-in per department in v1.1.

Decision	Status	Reason
Audit log scheduled export	v1.1	Manual CSV export available. Automation is a quality-of-life improvement, not a launch blocker.
Mobile app	Not planned	Browser-based SPA with responsive design is sufficient for intranet use on desktops/tablets.

ISTRAC-FMS: Chapter 14 — Testing Strategy

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 14: Testing Strategy	1
14.1 Gap Description	1
14.2 Testing Pyramid	1
14.3 Unit Tests	2
14.4 Integration Tests	2
14.5 End-to-End (E2E) Tests	2
14.6 Security Testing	3
14.7 CI/CD Pipeline	3

Chapter 14: Testing Strategy

This is a new chapter in v2.1. The v2.0 document contained no testing strategy. For a system managing sensitive files with approval workflows and audit compliance, a defined testing approach is essential.

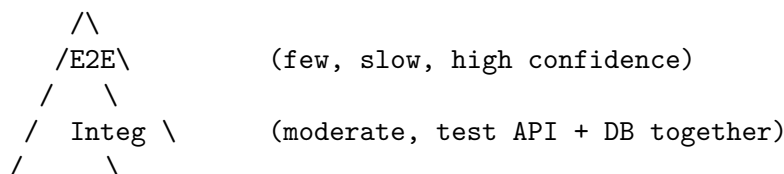
14.1 Gap Description

The v2.0 SRS contained no testing requirements, no framework choices, no coverage targets, and no CI/CD pipeline. This means:

- There is no automated safety net against regressions.
- The approval workflow, RBAC enforcement, and audit log immutability have no verified correctness.
- Deployments cannot be validated automatically before reaching staging or production.

14.2 Testing Pyramid

The project SHALL implement a standard testing pyramid with three layers:



/ Unit \
/-----\
 (many, fast, test pure logic)

14.3 Unit Tests

SHALL: All service-layer business logic SHALL have unit test coverage. Service tests SHALL use dependency injection to mock the Repository and HddService layers.

Framework: Jest (TypeScript, backend) + Vitest (TypeScript, frontend).

Coverage target: Minimum 80% line coverage on `src/services/` and `src/middleware/`.

Key test scenarios:

- **AuthService:** token generation, bcrypt validation, password policy enforcement, logout logic.
- **FileService:** upload compensation logic (HDD write fails, DB fails), checksum validation, version conflict detection.
- **SearchService:** result scoping — a query MUST NOT return files from departments the user is not assigned to.
- **AuditService:** every defined action type produces a correctly structured log record.
- **RBAC middleware:** all role/permission combinations in the permission matrix (Ch. 4.2) are tested.

SHALL: Unit tests SHALL run in under 30 seconds total. They SHALL mock all I/O (DB, HDD, email).

14.4 Integration Tests

SHALL: API integration tests SHALL test each REST endpoint against a real MySQL test database (spun up via Docker or a dedicated test schema).

Framework: Jest + Supertest (HTTP assertions against the Express app).

Test database: A separate MySQL schema (`istrac_fms_test`) seeded before each test run via migration + seed scripts.

Key test scenarios:

- Full upload → download → delete cycle with integrity check.
- Registration → approval → login → forced password change flow.
- Chunked upload: partial upload, resume from chunk 3, complete assembly.
- Soft delete: file marked deleted → inaccessible to members → recoverable by admin.
- Version control: two uploads of same filename → version 2 created → version 1 downloadable.
- Rate limit: 101st download within the hour returns 429.
- Pagination: list endpoint with 150 records returns exactly 50 items per page.

SHALL: Integration tests SHALL be isolated: each test or test suite SHALL clean up its own test data. No test SHALL depend on state left by another.

14.5 End-to-End (E2E) Tests

SHALL: Critical user journeys SHALL be covered by E2E tests using Playwright running against a staging environment.

Key E2E scenarios:

1. **New user onboarding:** Register → receive approval email → log in with temp password → change password → browse files.
2. **File upload & preview:** Upload a PDF → preview renders in browser → download and verify checksum.
3. **Admin approval workflow:** Submit registration → admin approves → user accesses assigned department.
4. **Search:** Upload a tagged file → search for the tag → result appears scoped correctly.
5. **Super Admin CMS edit:** Edit hero title → change visible on landing page without page reload.

SHALL: E2E tests SHALL run in CI against the staging environment on every merge to the `main` branch.

14.6 Security Testing

SHALL: Before each major release, a security review SHALL be performed covering:

- RBAC bypass attempts: direct API calls with insufficient role tokens.
- Path traversal probe: requests with `../` sequences in any user-supplied field.
- Upload bypass: attempt to upload a file with spoofed extension (e.g., `.jpg` with PHP magic bytes).
- CSRF probe: attempt to trigger a state-changing action from a cross-origin context.

SHOULD: A dependency vulnerability scan (`npm audit`) SHOULD run automatically in CI and block the build on critical vulnerabilities.

14.7 CI/CD Pipeline

SHALL: A CI pipeline SHALL be configured (GitHub Actions or GitLab CI) that runs on every pull request:

PR created / updated

```
Lint (ESLint + Prettier check)
Type check (tsc --noEmit)
Unit tests (Jest/Vitest) + coverage report
Integration tests (against test DB)
```

All pass → PR can be merged to main

Merge to main

```
Build production bundle
E2E tests against staging
Manual approval gate
```

Deploy to production

SHALL: The pipeline SHALL block merges to `main` if unit test coverage on `src/services/` drops below 80%.

SHALL: Build artifacts (compiled `dist/` folder) SHALL be produced by the CI pipeline and deployed to staging/production — never compiled directly on the production server.

ISTRAC-FMS: Chapter 15 — Gaps Summary, Traceability & Glossary

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 15: Gaps Summary, Traceability & Glossary	1
15.1 Consolidated Gaps Summary	1
15.2 Requirements Traceability Matrix	2
15.3 Glossary	4

Chapter 15: Gaps Summary, Traceability & Glossary

15.1 Consolidated Gaps Summary

This section summarizes every gap identified in v2.0 and addressed in v2.1, with its severity and the chapter where it is fully specified.

#	Gap	Severity	Chapter
G01	No LDAP / Active Directory integration	Critical	Ch. 10.6
G02	No download rate limiting	Critical	Ch. 10.4
G03	No proactive HDD mount health check	Critical	Ch. 10.5
G04	No testing strategy or CI/CD pipeline	Critical	Ch. 14
G05	No transactional integrity / rollback on upload	Critical	Ch. 3.5
G06	No concurrent write handling / file locking	Critical	Ch. 3.6
G07	No CSRF protection	Important	Ch. 7.2.2
G08	No HTTP security headers	Important	Ch. 7.2.1
G09	No CORS policy defined	Important	Ch. 7.2.3
G10	No password reset flow	Important	Ch. 5.5
G11	No password policy	Important	Ch. 5.6
G12	No MFA	Important	Ch. 5.7
G13	No API pagination	Important	Ch. 9.3
G14	No WebSocket authentication or reconnection	Important	Ch. 12.4
G15	No application observability / monitoring	Important	Ch. 7.5
G16	No folder management permissions defined	Important	Ch. 4.5
G17	No admin runtime configuration panel	Important	Ch. 4.6
G18	No disaster recovery plan or RTO/RPO	Important	Ch. 11.6

#	Gap	Severity	Chapter
G19	No staging / dev environment defined	Important	Ch. 2.4
G20	Tags stored as comma-separated VARCHAR	Minor	Ch. 8.3
G21	No chunked / resumable upload	Minor	Ch. 6.1.2
G22	No bulk file operations	Minor	Ch. 6.6
G23	No intranet file sharing links	Minor	Ch. 6.7
G24	No file encryption at rest	Minor	Ch. 7.2.4

15.2 Requirements Traceability Matrix

Requirement ID	Description	Chapter	Component
REQ-AUTH-01	Dual-token JWT strategy	Ch. 5.2	AuthService, refresh_tokens table
REQ-AUTH-02	Temp password + forced change on first login	Ch. 5.1	AuthService, users table
REQ-AUTH-03	Password reset via time-limited email token	Ch. 5.5	AuthService, EmailService
REQ-AUTH-04	Password policy (10 chars, complexity, history)	Ch. 5.6	AuthService, password_history table
REQ-AUTH-05	TOTP MFA (mandatory for Super Admin)	Ch. 5.7	AuthService, users.mfa_secret
REQ-AUTH-06	LDAP integration via Passport.js	Ch. 10.6	AuthService
REQ-FILE-01	Single-shot upload with magic byte validation	Ch. 6.1.1	FileService, HddService
REQ-FILE-02	SHA-256 checksum stored and verified on download	Ch. 3.5	FileService, hdd_nodes.sha256
REQ-FILE-03	Chunked / resumable upload	Ch. 6.1.2	FileService, upload_sessions table
REQ-FILE-04	Soft delete with 30-day recovery	Ch. 6.1.4	FileService, HddService
REQ-FILE-05	Version control on same-name upload	Ch. 6.1.5	FileService, file_versions table
REQ-FILE-06	Bulk operations (delete, tag, move)	Ch. 6.6	FileService, API
REQ-FILE-07	Intranet share links with expiry	Ch. 6.7	FileService, API

Requirement ID	Description	Chapter	Component
REQ-RBAC-01	Five-level role hierarchy enforced via middleware	Ch. 4.1, 4.2	RBAC middleware
REQ-RBAC-02	Per-department access levels	Ch. 4.3	user_department_access table
REQ-RBAC-03	Folder creation permissions	Ch. 4.5	RBAC middleware, hdd_nodes
REQ-RBAC-04	Admin runtime config panel	Ch. 4.6	AdminController, system_config table
REQ-SEC-01	HTTP security headers via Nginx	Ch. 7.2.1	Nginx config
REQ-SEC-02	CSRF protection (SameSite + header token)	Ch. 7.2.2	Auth middleware, cookie config
REQ-SEC-03	CORS restricted to app origin	Ch. 7.2.3	Express CORS middleware
REQ-SEC-04	Download rate limiting with anomaly alert	Ch. 10.4	RateLimit middleware, EmailService
REQ-SEC-05	HDD health check with instant admin alert	Ch. 10.5	HddHealthService, EmailService
REQ-API-01	Pagination on all list endpoints	Ch. 9.3	All list controllers
REQ-API-02	Standard error response envelope	Ch. 9.4	Express error handler
REQ-WS-01	WebSocket JWT authentication on connect	Ch. 12.4	WebSocket server
REQ-WS-02	Scoped broadcasts per department	Ch. 12.4	WebSocket server
REQ-WS-03	Reconnection with exponential backoff	Ch. 12.4	React WebSocket client
REQ-OBS-01	Structured JSON logging (pino)	Ch. 7.5	All services
REQ-OBS-02	/api/v1/health endpoint	Ch. 7.5	HealthController
REQ-TEST-01	80% unit test coverage on services	Ch. 14.3	Jest, src/services/
REQ-TEST-02	Integration tests per API endpoint	Ch. 14.4	Jest + Supertest
REQ-TEST-03	E2E tests for critical user journeys	Ch. 14.5	Playwright
REQ-TEST-04	CI pipeline blocking on failing tests	Ch. 14.7	GitHub Actions / GitLab CI

Requirement ID	Description	Chapter	Component
REQ-DR-01	Documented DR procedure with RPO 24h / RTO 4h	Ch. 11.6	Operations runbook
REQ-ENV-01	Three named environments (dev/staging/prod)	Ch. 2.4	Infrastructure

15.3 Glossary

Term	Definition
SPA	Single-Page Application — the React frontend loads once and updates dynamically without full page reloads.
HDD	Hard Disk Drive — the physical or networked storage where actual file bytes are stored.
NAS	Network-Attached Storage — a storage device accessible over the intranet network.
JWT	JSON Web Token — a signed token used to authenticate API requests. Contains user ID and role.
RBAC	Role-Based Access Control — permissions are granted based on user role, not individual user identity.
RHEL	Red Hat Enterprise Linux — the server operating system used to host all application components.
Soft Delete	Marking a file as deleted in the database without immediately removing it from disk, allowing recovery.
Audit Log	An immutable, append-only record of every action taken in the system — who did what, when.
<code>hdd_nodes</code>	The database table that mirrors the physical HDD file tree with metadata and permissions.
Orphaned	A file that exists in the database but has been removed from the HDD outside of the application.
Unregistered	A file found on the HDD by the sync daemon that has no corresponding database record yet.
CMS	Content Management System — the admin-editable landing page content stored in the database.
SMTP	Simple Mail Transfer Protocol — used for sending campus-internal email notifications.

Term	Definition
Magic Bytes	The first few bytes of a file that identify its true type, used to prevent extension spoofing.
PM2	A Node.js process manager that keeps the server running and restarts it on crash or reboot.
TOTP	Time-based One-Time Password — RFC 6238 standard for MFA codes generated by authenticator apps.
LDAP	Lightweight Directory Access Protocol — the protocol used to authenticate against campus Active Directory.
CSRF	Cross-Site Request Forgery — an attack that tricks a user's browser into making unauthorized requests.
CORS	Cross-Origin Resource Sharing — a browser mechanism controlling which origins can access API resources.
RPO	Recovery Point Objective — the maximum acceptable amount of data loss in a disaster scenario.
RTO	Recovery Time Objective — the maximum acceptable time to restore the system after a failure.
SHA-256	Secure Hash Algorithm — a cryptographic hash used to verify file integrity after storage.
Cursor Pagination	A pagination method using an opaque cursor instead of page numbers — stable on large append-only tables.
CSP	Content Security Policy — an HTTP header that restricts which resources the browser can load.

ISTRAC-FMS SRS & Architecture v2.1 / Confidential — Internal Use Only

This document supersedes v2.0 dated June 2026. All gaps identified in v2.0 Section 14 are carried forward and addressed herein.

ISTRAC-FMS: Chapter 16 — Custom Role System

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 16: Custom Role System	1
16.1 Concept: Custom Roles on Top of Fixed Hierarchy	1
16.2 Permission Keys and Schema	2
16.3 Permission Resolution — Five-Level Logic	3
16.4 API Routes for Custom Roles	5
16.5 Admin Dashboard — Custom Role Management UI	6

Chapter 16: Custom Role System

New in v2.1. This chapter specifies the custom role and permission system that sits on top of the fixed four-tier hierarchy (Super Admin / Dept Admin / Member / Guest). It covers the permission model, database schema, API surface, and the `PermissionService` resolution logic.

16.1 Concept: Custom Roles on Top of Fixed Hierarchy

The base system uses four fixed roles (L1–L3r) that control coarse-grained access. Custom roles are an **additive layer** that allow Super Admins and Dept Admins to define fine-grained permission sets and assign them to users within a department — without changing a user’s base role.

Key design principles:

- **Additive, not replacement.** Custom roles grant additional permissions on top of what the base role already allows. They do not remove base-role permissions (use DENY overrides for that — see §16.3).
- **Department-scoped.** A custom role exists within a specific department. The same named role in two departments is independent and may have different permissions.
- **Delegatable.** Super Admin can grant Dept Admins the right to create and assign custom roles within their own department (controlled by the `MANAGE_ROLES` permission key).
- **Composable.** A user can hold multiple custom roles simultaneously within a department. Permissions are unioned across all assigned custom roles.

Resolution order (highest wins, DENY always beats GRANT):

1. Per-user DENY override (explicit denial, always wins)
2. Per-user GRANT override (explicit grant for one user)

- 3. Custom role permission (any) (unioned across all assigned roles)
- 4. Base role default (from fixed hierarchy)
- 5. System default (implicit DENY for anything not listed)

16.2 Permission Keys and Schema

16.2.1 Atomic Permission Keys

The system defines **20 atomic permission keys**. Every feature that can be selectively enabled or disabled maps to exactly one key. These keys are the vocabulary used in custom roles and per-user overrides.

#	Permission Key	Description
1	FILE_VIEW	View file metadata within the department
2	FILE_DOWNLOAD	Download file bytes
3	FILE_UPLOAD	Upload new files
4	FILE_DELETE	Soft-delete files (move to trash)
5	FILE_RESTORE	Restore files from trash
6	FILE_VERSION	Access and download version history
7	FILE_SHARE	Generate intranet share links
8	FILE_TAG	Add or remove tags on files
9	FILE_BULK	Perform bulk operations (bulk delete, tag, move)
10	FOLDER_CREATE	Create subfolders
11	FOLDER_RENAME	Rename folders
12	FOLDER_DELETE	Delete folders (soft-delete all contents)
13	USER_APPROVE	Approve or reject pending user registrations
14	USER_SUSPEND	Suspend or reinstate user accounts
15	AUDIT_VIEW	View the department audit log
16	AUDIT_EXPORT	Export audit log as CSV
17	CMS_EDIT	Edit CMS landing page content
18	MANAGE_ROLES	Create, edit, and assign custom roles within own department
19	DEPT_SETTINGS	Edit department name, HDD path, and configuration
20	SEARCH_CROSS_DEPT	Search across departments (Super Admin only by default)

16.2.2 Database Schema

```
-- Custom role definitions (dept-scoped)
custom_roles:
  id          CHAR(36)      PK DEFAULT (UUID())
  dept_id     CHAR(36)      FK → departments(id)  -- NULL = system-wide (Super Admin only)
  name        VARCHAR(100) NOT NULL
  description  TEXT
  permissions  JSON         NOT NULL -- array of PermissionKey strings, e.g. ["FILE_UPLOAD", "I
  created_by   CHAR(36)      FK → users(id)
  created_at   DATETIME     DEFAULT NOW()
  updated_at   DATETIME     DEFAULT NOW() ON UPDATE NOW()
  UNIQUE (dept_id, name)

-- Assignments: which users hold which custom roles (dept-scoped)
user_custom_roles:
  id          CHAR(36)      PK DEFAULT (UUID())
  user_id     CHAR(36)      FK → users(id)
  role_id     CHAR(36)      FK → custom_roles(id)
  dept_id     CHAR(36)      FK → departments(id)
  assigned_by  CHAR(36)      FK → users(id)
  assigned_at  DATETIME     DEFAULT NOW()
  UNIQUE (user_id, role_id, dept_id)

-- Per-user, per-permission GRANT/DENY overrides (dept-scoped)
user_permission_overrides:
  id          CHAR(36)      PK DEFAULT (UUID())
  user_id     CHAR(36)      FK → users(id)
  dept_id     CHAR(36)      FK → departments(id)
  permission  VARCHAR(50)   NOT NULL -- one of the 20 PermissionKey values
  effect      ENUM('GRANT', 'DENY') NOT NULL
  granted_by   CHAR(36)      FK → users(id)
  granted_at   DATETIME     DEFAULT NOW()
  UNIQUE (user_id, dept_id, permission)
```

16.2.3 user_dashboard_layouts Table

Admins can customise their dashboard widget layout. Preferences are persisted per user.

```
user_dashboard_layouts:
  user_id     CHAR(36)      PK FK → users(id)
  layout_json  JSON         NOT NULL -- widget positions, sizes, and visibility flags
  updated_at   DATETIME     DEFAULT NOW() ON UPDATE NOW()
```

16.3 Permission Resolution — Five-Level Logic

Every access check goes through `PermissionService.can(userId, deptId, permissionKey)`. The resolution runs top-to-bottom and short-circuits at the first definitive answer.

```

async function can(
  userId: string,
  deptId: string,
  key: PermissionKey
): Promise<boolean> {

  // Level 1: Per-user DENY override - always wins
  const denyOverride = await OverrideRepo.find(userId, deptId, key, 'DENY');
  if (denyOverride) return false;

  // Level 2: Per-user GRANT override
  const grantOverride = await OverrideRepo.find(userId, deptId, key, 'GRANT');
  if (grantOverride) return true;

  // Level 3: Any assigned custom role has this permission
  const customRoles = await CustomRoleRepo.findForUser(userId, deptId);
  const hasViaRole = customRoles.some(r => r.permissions.includes(key));
  if (hasViaRole) return true;

  // Level 4: Base role default (from fixed hierarchy)
  const baseRole = await UserRepo.getBaseRole(userId);
  if (BASE_ROLE_DEFAULTS[baseRole].includes(key)) return true;

  // Level 5: Implicit deny
  return false;
}

```

DENY always wins. If a user has a DENY override for FILE_DOWNLOAD, they cannot download even if their custom role or base role grants it. This allows admins to restrict specific users without changing their base role (e.g., temporarily restricting a user under review).

Formal Requirements

SHALL: `PermissionService.can()` SHALL be the single point of permission resolution. No controller or service SHALL perform inline role checks outside of this service.

SHALL: Permission checks SHALL be cached per-request (within a single HTTP request lifecycle) to avoid redundant DB lookups when a route calls `can()` multiple times.

SHALL: All custom role mutations (create, update, delete, assign, revoke) SHALL be logged to `audit_logs` with `old_value` (previous permissions JSON) and `new_value`.

SHALL: Deleting a custom role SHALL cascade: all `user_custom_roles` records referencing that role SHALL be deleted, and affected users SHALL receive a notification that a role was removed from them.

SHALL: A user cannot be assigned a custom role that grants permissions beyond what the assigning admin's own effective permission set allows (privilege escalation prevention).

16.4 API Routes for Custom Roles

11 new routes are added to the API surface for role CRUD, assignment, and per-user overrides.

Method + Path	Auth	Role Gate	Description
GET /api/v1/roles?deptId=	JWT	MANAGE_ROLES SUPER_ADMIN	List custom roles for a department
POST /api/v1/roles	JWT	MANAGE_ROLES SUPER_ADMIN	Create a new custom role
GET /api/v1/roles/:id	JWT	MANAGE_ROLES SUPER_ADMIN	Get role details and permission list
PUT /api/v1/roles/:id	JWT	MANAGE_ROLES SUPER_ADMIN	Update role name, description, or permissions
DELETE /api/v1/roles/:id	JWT	SUPER_ADMIN	Delete role (cascade removes assignments)
GET /api/v1/roles/:id/members	JWT	MANAGE_ROLES SUPER_ADMIN	List users assigned to this role
POST /api/v1/roles/:id/assign	JWT	MANAGE_ROLES SUPER_ADMIN	Assign role to a user
DELETE /api/v1/roles/:id/assign/:userId	JWT	MANAGE_ROLES SUPER_ADMIN	Remove role from a user
GET /api/v1/users/:id/permissions?deptId=	JWT	MANAGE_ROLES SUPER_ADMIN	Get a user's resolved effective permissions
POST /api/v1/users/:id/permissions/override	JWT	SUPER_ADMIN	Add a GRANT or DENY override for a user
DELETE /api/v1/users/:id/permissions/override	JWT	SUPER_ADMIN	Remove a specific permission override

Request/Response Shapes

Create role request:

POST /api/v1/roles

```
{
  "deptId": "uuid",
  "name": "Senior Archivist",
  "description": "Can tag, version, and share files but not delete",
  "permissions": ["FILE_VIEW", "FILE_DOWNLOAD", "FILE_UPLOAD", "FILE_VERSION", "FILE_TAG", "FILE_SHARE"]
}
```

Effective permissions response:

GET /api/v1/users/:id/permissions?deptId=uuid

```
{
  "userId": "uuid",
  "deptId": "uuid",
  "baseRole": "MEMBER",
  "permissions": {
    "FILE_VIEW": true,
    "FILE_DOWNLOAD": true,
    "FILE_UPLOAD": true,
    "FILE_VERSION": true,
    "FILE_TAG": true,
    "FILE_SHARE": true
  }
}
```

```

"customRoles": ["Senior Archivist"],
"overrides": [
  { "permission": "FILE_DELETE", "effect": "DENY" }
],
"effectivePermissions": ["FILE_VIEW", "FILE_DOWNLOAD", "FILE_UPLOAD", "FILE_VERSION", "FILE_TAG"],
"deniedPermissions": ["FILE_DELETE"]
}

```

16.5 Admin Dashboard — Custom Role Management UI

The admin role management interface is accessible at `/admin/roles` (gated by `MANAGE_ROLES` permission).

Dashboard widgets (8 total):

Widget	Description
Role List	Table of all custom roles in the department, with member count and permission summary
Role Editor	Create or edit a role — checkbox grid of all 20 permission keys with description tooltips
Role Members	List of users assigned to the selected role, with assign/revoke controls
User Permission Inspector	Search for any user, view their full resolved effective permissions across all assigned roles and overrides
Override Panel	Add GRANT or DENY overrides for a specific user + permission combination
Permission Audit Trail	Recent audit log filtered to role and permission events
Role Usage Stats	Which roles are most widely assigned; which permissions are most commonly overridden
Quick Assign	Typeahead search for a user → select from existing roles → assign in one click

Layout persistence: Widget positions and visibility are saved per-user to the `user_dashboard_layouts` table. On next login the dashboard restores to the user's last saved arrangement.

Formal Requirements

SHALL: The role editor SHALL display all 20 permission keys as a checklist, grouped into categories (File Operations, Folder Operations, User Management, Audit & Export, System).

SHALL: Saving a role in the editor SHALL call `PUT /api/v1/roles/:id` and optimistically update the UI via React Query cache invalidation.

SHALL: The User Permission Inspector SHALL show the resolution chain: which permission came from which source (base role, custom role name, or override), so admins can understand why a user has or lacks a specific permission.

SHALL: Dashboard layout changes SHALL be auto-saved (debounced 2 seconds) to PUT `/api/v1/admin/dashboard/layout` without requiring an explicit save button.

ISTRAC-FMS: Chapter 17 — In-App Notification System

Software Requirements Specification & System Architecture v2.1

ISTRAC Engineering

June 2026

Contents

Chapter 17: In-App Notification System	1
17.1 Overview	1
17.2 Notification Event Types	1
17.3 Backend Event Pipeline	2
17.4 Database Schema	3
17.5 Toast Notification Component	4
17.6 Notification Inbox (/notifications)	5
17.7 Notification API Routes	6
17.8 Integration Points	7

Chapter 17: In-App Notification System

New in v2.1. This chapter specifies the full in-app notification system: the backend event pipeline, database schema, WebSocket delivery, toast component, the /notifications page, and the addNotification() global API available to all application features.

17.1 Overview

The notification system delivers real-time and persisted messages to users for every significant event in the system. It has two delivery channels:

- **Toast notifications:** Ephemeral pop-up messages shown immediately in the current session, auto-dismissing after a configurable duration.
- **Notification inbox (/notifications):** A persistent, browsable history of all notifications, with read/unread state, filtering, and bulk management.

Both channels are driven from the same backend event pipeline. Toasts are delivered via WebSocket to active sessions; inbox items are persisted to the notifications DB table and available to users who were offline when the event occurred.

17.2 Notification Event Types

Event Type	Trigger	Recipients	Category
FILE_UPLOAD	A file is uploaded to a department	All active members of that department	Files
FILE_DELETED	A file is soft-deleted	Dept Admin + file uploader	Files
FILE_RESTORED	A file is restored from trash	Dept Admin + file uploader	Files
FILE_VERSION	A new version of a file is created	Users who have favorited the file	Files
FILE_SHARED	A share link is generated for a file	File uploader (if not self)	Files
APPROVAL_PENDING	A new registration is submitted	All Dept Admins + Super Admin	Approvals
APPROVAL_APPROVED	A registration is approved	The registering user	Approvals
APPROVAL_REJECTED	A registration is rejected	The registering user	Approvals
USER_SUSPENDED	A user account is suspended	The suspended user	System
USER_ROLE_ASSIGNED	A custom role is assigned to a user	The assigned user	System
USER_ROLE_REMOVED	A custom role is removed from a user	The affected user	System
GUEST_EXPIRY_7D	Guest access expires in 7 days	The guest user	System
GUEST_EXPIRY_1D	Guest access expires in 1 day	The guest user	System
HDD_SYNC_COMPLETE	HDD sync daemon finishes a run	Super Admin only	System
HDD_UNAVAILABLE	HDD mount health check fails	Super Admin only	System
HDD_RECOVERED	HDD mount health check recovers	Super Admin only	System
DOWNLOAD_RATE_LIMIT	A user hits the download rate limit	Super Admin (anomaly alert)	System
CMS_UPDATED	Admin updates CMS content	No notification (silent WebSocket push)	—

17.3 Backend Event Pipeline

All notifications flow through `NotificationService`, which is the single entry point for creating notification records and dispatching WebSocket messages.

```
// NotificationService.send() - called by any service after a triggering action
async function send(event: {
  type: NotificationEventType;
  actorId: string;           // user who triggered the event
```



```

recipientIds: string[]; // who receives it
resourceType: string; // 'file' | 'user' | 'department'
resourceId: string;
message: string; // human-readable message
metadata?: Record<string, unknown>; // extra context (file name, dept name, etc.)
}): Promise<void> {
  // 1. Persist to DB for each recipient
  await NotificationRepo.insertMany(event.recipientIds.map(userId => ({
    userId,
    type: event.type,
    actorId: event.actorId,
    resourceType: event.resourceType,
    resourceId: event.resourceId,
    message: event.message,
    metadata: event.metadata,
    readAt: null,
    createdAt: new Date(),
  })));

  // 2. Push to active WebSocket sessions for each recipient
  for (const userId of event.recipientIds) {
    WebSocketServer.sendToUser(userId, {
      channel: 'notification',
      payload: { type: event.type, message: event.message, metadata: event.metadata },
    });
  }
}

```

Formal Requirements

SHALL: `NotificationService.send()` SHALL be the only mechanism for creating notification records. No controller or service SHALL write directly to the `notifications` table.

SHALL: Notification inserts SHALL NOT block the triggering action's response. They SHALL be executed asynchronously (fire-and-forget with error logging) after the primary operation is committed.

SHALL: If the WebSocket push fails for a recipient (user offline or socket closed), the notification SHALL remain persisted in the DB and be delivered on next page load via REST API.

17.4 Database Schema

notifications:

id	BIGINT	AUTO_INCREMENT PK
user_id	CHAR(36)	FK → users(id)
type	VARCHAR(50)	NOT NULL -- NotificationEventType enum value
actor_id	CHAR(36)	FK → users(id) -- who triggered it (NULL for system events)
resource_type	VARCHAR(50)	NULL
resource_id	CHAR(36)	NULL

```

message      TEXT      NOT NULL
metadata     JSON      NULL      -- extra context (e.g. { "fileName": "report.pdf" })
read_at      DATETIME  NULL      -- NULL = unread
dismissed_at DATETIME  NULL      -- NULL = not dismissed from inbox
category     ENUM('Files','Approvals','System') NOT NULL
created_at   DATETIME  DEFAULT NOW()

INDEX idx_notif_user    (user_id, read_at, created_at)
INDEX idx_notif_type    (user_id, type, created_at)

```

Formal Requirements

SHALL: The notifications table SHALL use cursor-based pagination when queried (see Ch. 9.3). The default page size is 30 notifications per page.

SHALL: Notifications older than 90 days SHALL be automatically purged by a daily cron job. Purge events SHALL be logged to audit_logs.

SHALL: The unread count (used for the bell badge) SHALL be computed as a lightweight `SELECT COUNT(*)` on `(user_id, read_at IS NULL, dismissed_at IS NULL)` — not by fetching all notifications.

17.5 Toast Notification Component

The `ToastContainer` is mounted once in `AppShell` and is globally accessible. It renders a bottom-right stack of toast cards.

17.5.1 Toast Behaviour

Property	Value
Position	Bottom-right of viewport
Stack direction	Newest on top
Max visible	5 toasts simultaneously; overflow queued
Auto-dismiss	5 seconds (configurable per toast type)
Progress bar	Visible at bottom of each toast, depletes over auto-dismiss duration
Manual dismiss	Close button (×) on each toast
Pause on hover	Auto-dismiss timer pauses when mouse is over the toast

17.5.2 Toast Variants

Variant	Used For	Color
success	File uploaded, approval approved, restore complete	Green
info	New file in department, role assigned	Blue
warning	Guest access expiry, rate limit approaching	Amber
error	HDD unavailable, upload failed	Red

17.5.3 addNotification() Global API

Any component or hook in the application can trigger a toast via the global `useApp()` hook:

```
const { addNotification } = useApp();
```

```
// Trigger a toast from anywhere in the app
addNotification({
  type: 'success',
  title: 'File Uploaded',
  message: 'report_q2.pdf was uploaded to SIMS.',
  duration: 5000,           // ms, optional - defaults to 5000
  action: {                 // optional action button in the toast
    label: 'View',
    onClick: () => navigate(`/files/${fileId}`),
  },
});
```

SHALL: `addNotification()` SHALL be the only mechanism for triggering toasts. Components SHALL NOT render toast elements directly.

SHALL: WebSocket notifications received from the server SHALL automatically call `addNotification()` through a central WebSocket message handler in `AppShell`, ensuring all server-pushed events appear as toasts.

17.6 Notification Inbox (/notifications)

The full notification history is accessible at `/notifications`, linked from “View all notifications →” in the Topbar dropdown.

17.6.1 Page Layout

`/notifications`

Header: "Notifications" + "Mark all read" button + "Clear read" button

Tab bar: All | Unread | Files | System | Approvals

(tab labels show badge count for unread in that category)

Notification list (paginated, cursor-based)

Date group header: "Today", "Yesterday", "This Week", older dates

Notification card:

Icon (category-specific)

Message text (with bold resource name)

Relative timestamp ("2 hours ago")

"Mark read" / "Mark unread" toggle

Dismiss button (removes from inbox view)

Action link (e.g. "View file →") if resource still exists

... more cards

Load more (infinite scroll, fetches next cursor page)

17.6.2 Tabs

Tab	Filter
All	All non-dismissed notifications
Unread	<code>read_at</code> IS NULL only
Files	<code>category</code> = 'Files'
System	<code>category</code> = 'System'
Approvals	<code>category</code> = 'Approvals'

17.6.3 Bell Badge in Sidebar

The bell icon in the Sidebar shows an unread count badge. The count is fetched on mount and updated in real-time via WebSocket (each incoming notification increments the count in Zustand state without a network round-trip).

```
// Zustand slice for notification state
interface NotificationState {
  unreadCount: number;
  incrementUnread: () => void;
  decrementUnread: (by: number) => void;
  resetUnread: () => void;
}
```

The Topbar bell dropdown shows the 5 most recent unread notifications with a “View all →” link to `/notifications`.

17.6.4 Formal Requirements

SHALL: The `/notifications` page SHALL load the first page of notifications on mount via `GET /api/v1/notifications` (cursor-based, 30 per page).

SHALL: “Mark all read” SHALL call `POST /api/v1/notifications/mark-all-read`, set `read_at` = `NOW()` for all unread notifications for the current user, and reset the bell badge count to 0.

SHALL: “Clear read” SHALL call `POST /api/v1/notifications/dismiss-read`, set `dismissed_at` = `NOW()` for all read notifications, and remove them from the inbox view.

SHALL: Dismissing a single notification SHALL call `DELETE /api/v1/notifications/:id` and optimistically remove it from the list.

SHALL: The notification bell badge SHALL be visible in the Sidebar on all authenticated pages. An unread count of 0 SHALL hide the badge number (not show “0”).

SHALL: Clicking a notification card’s action link SHALL first mark the notification as read, then navigate to the resource.

SHALL: If a notification’s linked resource no longer exists (file deleted, user removed), the action link SHALL be hidden and the message text SHALL indicate the resource is no longer available.

17.7 Notification API Routes

Method + Path	Auth	Description
GET /api/v1/notifications	JWT	Get paginated inbox (supports tab filter via <code>?category=&unread=</code>)
GET /api/v1/notifications/count	JWT	Get unread count (lightweight — for badge)
POST /api/v1/notifications/mark-all-read	JWT	Mark all unread as read
PATCH /api/v1/notifications/:id/read	JWT	Mark single notification as read
PATCH /api/v1/notifications/:id/unread	JWT	Mark single notification as unread
DELETE /api/v1/notifications/:id	JWT	Dismiss (hide) a single notification
POST /api/v1/notifications/dismiss-read	JWT	Dismiss all read notifications

17.8 Integration Points

Every service that triggers a notification SHALL call `NotificationService.send()` after its primary DB operation succeeds. The following integration table maps triggering services to notification events:

Service Method	Event Type	Recipients
<code>FileService.upload()</code>	<code>FILE_UPLOAD</code>	All dept members (opt-in configurable)
<code>FileService.softDelete()</code>	<code>FILE_DELETED</code>	Dept Admin + uploader
<code>FileService.restore()</code>	<code>FILE_RESTORED</code>	Dept Admin + uploader
<code>FileService.createVersion()</code>	<code>FILE_VERSION</code>	Users who favorited the file
<code>UserService.approve()</code>	<code>APPROVAL_APPROVED</code>	Registering user
<code>UserService.reject()</code>	<code>APPROVAL_REJECTED</code>	Registering user
<code>UserService.submitRegistration()</code>	<code>APPROVAL_PENDING</code>	All Dept Admins + Super Admin
<code>UserService.suspend()</code>	<code>USER_SUSPENDED</code>	Suspended user
<code>CustomRoleService.assign()</code>	<code>USER_ROLE_ASSIGNED</code>	Assigned user
<code>CustomRoleService.revoke()</code>	<code>USER_ROLE_REMOVED</code>	Affected user
<code>HddHealthService (cron)</code>	<code>HDD_UNAVAILABLE / HDD_RECOVERED</code>	Super Admin
<code>HddSyncService.run()</code>	<code>HDD_SYNC_COMPLETE</code>	Super Admin
<code>GuestExpiryJob (cron)</code>	<code>GUEST_EXPIRY_7D / GUEST_EXPIRY_1D</code>	Guest user

SHALL: File upload notifications SHALL be opt-in per user (controlled by a user preference flag `notify_on_dept_upload`). Users who have opted out SHALL NOT receive `FILE_UPLOAD` notifications, but all other notification types remain mandatory.

SHALL: The notification system SHALL degrade gracefully: if `NotificationService.send()` throws, the error SHALL be caught, logged, and the triggering service's response SHALL not be affected. A failed notification SHALL never cause a transaction rollback.